Class 6 - **02 November 2026**

Build & Improving GenAI systems

Introduction to AI and Machine Learning

Executive Master in Artificial Intelligence for Business

Roberto Henriques · Fernando Peres

NOVA IMS INFORMATION MANAGEMENT SCHOOL

Table of Contents

01. What is a Harness?

02. The Mindset for GenAI Systems

03. Context & Context Engineering

04. Tools and Integration

05. Orchestrators: Workflows and Agents

06. Guardrails and Evals

07. Observability

08. Experiments

09. Bonus: Cycles of Improvement

10. Project: Your GenAI System

01. What is a **harness**?

The model is the engine; the harness turns it into a system.

- What a harness is
- Model ≠ GenAI system
- Its components
- Implemented harnesses vs. frameworks

ONE IDEA TO KEEP

A model on its own is not a product. The harness decides what it can see, do and be trusted with.

Same model, different harness, different system.

LESSON

Harness overview

DEPTH

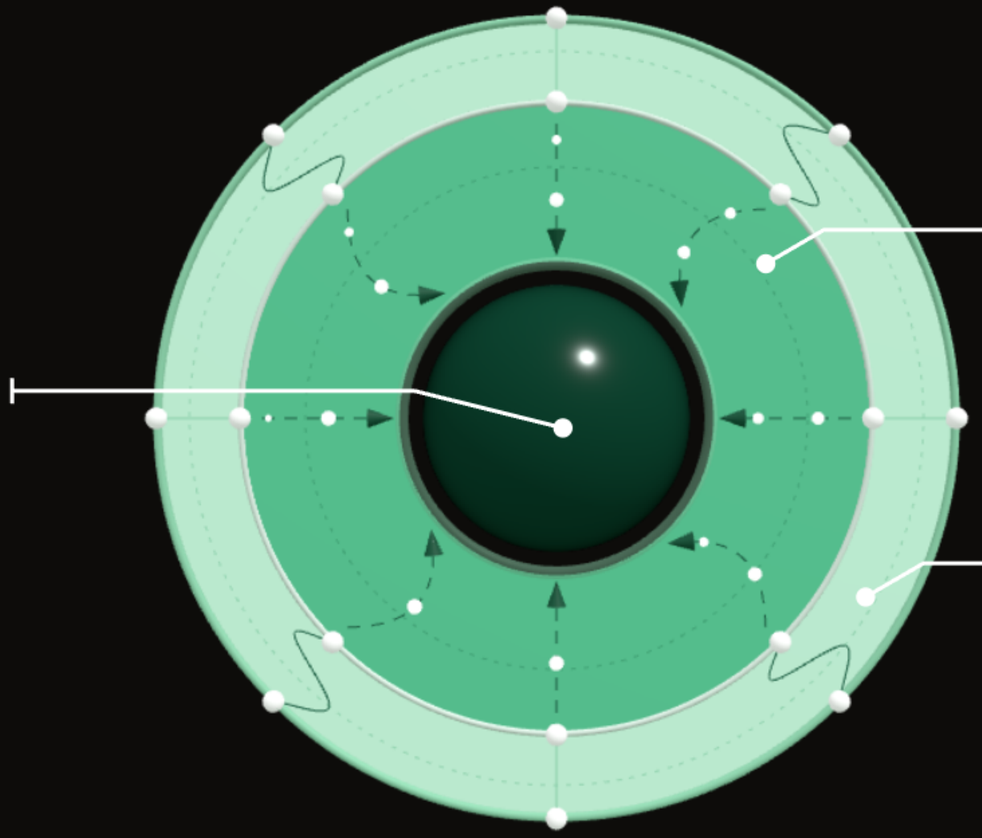
- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

Understand what a harness is: the layer around a GenAI model that contextualizes, extends, coordinates, constrains and operationalizes it. Tell apart the three layers of a GenAI system (the model, the harness, and the context and knowledge) and why the quality of the system depends on the harness as much as on the model.

Harness for GenAI systems capability extensions

A harness contextualizes, extends, coordinates, constrains, and operationalizes models to enable the development and execution of GenAI systems.*



The system that orchestrates and controls the intelligence.

HARNESS

MODEL

The intelligence that thinks and generates.

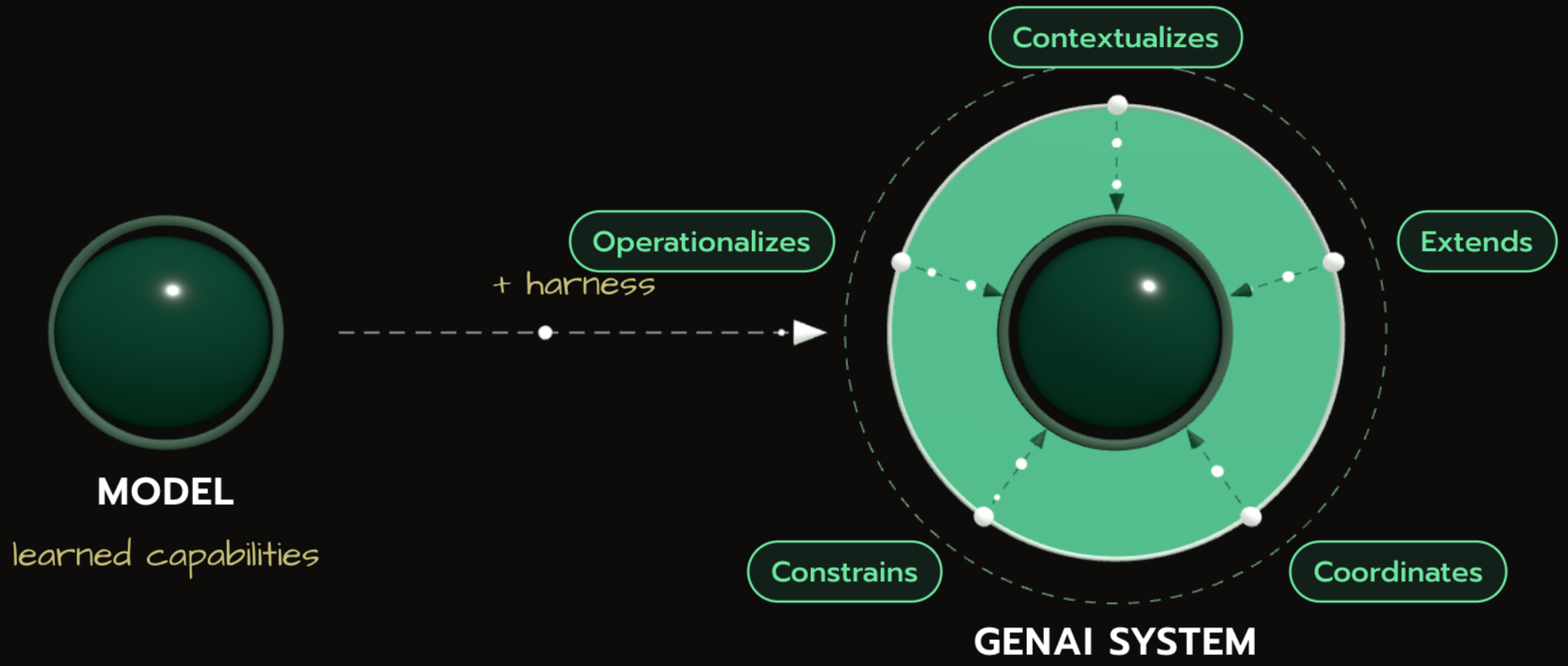
CONTEXT AND KNOWLEDGE

The information that grounds and guides the intelligence.

drag to rotate · double-click to reset

Model ≠ GenAI System

The model provides learned capabilities, while the harness contextualizes, extends, coordinates, constrains, and operationalizes them.



The components of a harness

A harness turns a model into a working system through five building blocks.

Context and knowledge integration	What the model sees: instructions, documents and retrieved data.
Memory and state management	What persists across steps, turns and sessions.
Tools and external capabilities	Actions beyond text: search, code execution and APIs.
Orchestration and control	Loops, workflows, routing, permissions and guardrails.
Evaluation mechanisms	Tests, metrics and feedback that check and improve outputs.

Three layers, three roles

1 · MODEL

Models are the intelligence

They provide the reasoning, understanding, and generation capabilities.

2 · HARNESS

The harness is the system around the intelligence

It orchestrates models, tools, context, and workflows to turn intelligence into reliable applications.

3 · CONTEXT & KNOWLEDGE

Context & knowledge provide what the intelligence needs

They ground the model in relevant data, memory, and knowledge to produce meaningful, context-aware responses.

Note: "intelligence" and "thinks" are shorthand. Models generate from learned patterns; they don't think the way people do. And context is more than knowledge: it also carries instructions, goals, and the current state of the task.

Implemented Harnesses — Ready-to-Use GenAI Systems

PRODUCT	PROVIDER	MAIN HARNESS CAPABILITIES
ChatGPT	OpenAI	Context management, memory, web search, tools, file processing, application integrations, agentic workflows
Claude	Anthropic	Conversational context, projects, artifacts, knowledge integration, tools, application connections
Gemini	Google	Multimodal interaction, tool use, information retrieval, Google ecosystem integrations
Microsoft Copilot	Microsoft	Microsoft 365 context, work and web grounding, document and email integration, agents
Perplexity	Perplexity AI	Real-time web search, source citation, retrieval, research workflows

Implemented Harnesses — Ready-to-Use GenAI Systems

PRODUCT	PROVIDER	MAIN HARNESS CAPABILITIES
OpenAI Codex	OpenAI	Coding-agent loop, repository context, terminal execution, testing, code editing, sandboxing, approvals
Claude Code	Anthropic	Agent loop, file-system and shell tools, project instructions, skills, hooks, MCP integrations, permissions
GitHub Copilot	GitHub/Microsoft	IDE context, repository integration, coding tools, agent workflows, code review
Cursor	Anysphere	Codebase indexing, file editing, terminal tools, agentic coding, workspace context
Replit	Replit	Cloud workspace, agent that plans and builds apps, code execution, databases, deployment
Lovable	Lovable	Prompt-to-app generation, full-stack scaffolding, visual editing, backend integrations, deployment

ChatGPT is a harness, but not a harness for coding

Before · a chat harness used for code

ChatGPT in a browser tab

- YOU — Describe the problem, paste code and errors into the chat
- ↓
- MODEL — Writes code as text in the chat
- ↓
- YOU — Read it, understand it, copy it into VS Code
- ↓
- YOU — Adapt it to your files, run it, test it
- ↓
- YOU — Copy the error back into the chat, and repeat ↺

The model never sees your project. **You** are its file system, terminal and test runner.

Now · a coding harness

Claude Code, Codex, Copilot agent mode

- YOU — Give the task, or point to a spec
- ↓
- HARNESS — Plans, reads and searches the repository
- ↓
- HARNESS — Creates and edits files, shows the diff
- ↓
- HARNESS — Runs commands and tests, reads errors, fixes, repeats ↺
- ↓
- YOU — Review the diff or pull request, approve

Same kind of model. The harness gives it **eyes and hands in your project**; you review instead of relaying.

Sources: Anthropic, Claude Code documentation; VS Code blog, *Introducing GitHub Copilot agent mode* (Feb 2025); OpenAI, Codex launch (May 2025).

What a coding harness builds around the model

Read & search the repo
was: paste code into the chat

Create & edit files
was: copy, paste and adapt

Run terminal, tests, linters
was: run it yourself, paste errors back

Plan before acting
plan mode, to-do lists · was: in your head

Project instructions & specs
AGENTS.md, CLAUDE.md · was: repeat it yourself

Permissions & sandbox
approve edits and commands · was: only you

Integrations
MCP, web search, git, pull requests

Where you work
terminal, IDE, desktop app, cloud

HARNESS · FRAMEWORKS

Harness Development Frameworks — Frameworks and SDKs for Building GenAI Systems

Implemented harnesses are products you use. These frameworks are the building blocks you use to make your own harness.

FRAMEWORK / SDK	ORGANIZATION	MAIN HARNESS COMPONENTS
LangChain	LangChain	Model integration, agents, tools, retrieval, middleware
LangGraph	LangChain	Stateful graphs, workflow orchestration, persistence, checkpoints, human-in-the-loop
CrewAI	CrewAI	Agents, roles, crews, tasks, flows, tools, delegation
OpenAI Agents SDK	OpenAI	Agent loop, tools, handoffs, guardrails, sessions, tracing
Microsoft AutoGen	Microsoft	Agent communication, multi-agent collaboration, message routing, orchestration
Semantic Kernel	Microsoft	Plugins, function calling, model connectors, process orchestration
LlamaIndex	LlamaIndex	Data connectors, indexing, RAG, retrieval, knowledge-centric agents

HARNESS · COMPARE

Implemented harnesses vs. harness development frameworks

One is a finished system ready to use; the other is a kit for building your own.

Dimension	Implemented harnesses	Harness development frameworks
Main purpose	Execute real user tasks	Build customized AI applications and agents
Target audience	End users, developers, organizations	AI engineers and software developers
Readiness	Operational system	Building blocks requiring configuration and implementation
Model integration	Already integrated	Developer-configured
Tools, memory, orchestration	Prebuilt or configurable	Supplied as primitives, abstractions, and APIs
Examples	ChatGPT, Claude Code, Codex	LangChain, LangGraph, CrewAI, OpenAI Agents SDK

02. The Mindset for **GenAI Systems**

Improve the whole system, continuously and with evidence.

Four dimensions | Limits of model optimization | The cost of training | Takeaways

ONE IDEA TO KEEP

Don’t wait for a better model. Improve the system around it, and let evidence decide what to change.

Most of the gains you control are outside the weights.

Improving the Performance of GenAI Systems

Four complementary dimensions form a continuous improvement framework for GenAI systems, addressing their limitations and improving robustness, reliability, and output quality.

1 Model Training and Adaptation

- Pre-training (and continued pre-training)
- Post-training (RLHF and other post-training techniques)

Parametric memory

2 Harness and Capability Extension

- Memory systems and artifacts
- Context and knowledge integration
- Capabilities and tools
- Orchestration and control
- Execution and evaluation

3 Experiments and Validation

(offline)

- Benchmarks
- Datasets
- Test cases
- Metrics
- Evaluations

4 Observability and Continuous Improvement

- Telemetry
- Traces
- Metrics and evaluators
- User feedback

Four Dimensions of Continuous Improvement

1 Model Training and Adaptation

Improve model capabilities, alignment, efficiency, and task-specific performance through model selection, configuration, and adaptation.

2 Harness and Capability Extension

Extend and control model behavior through context, knowledge, memory, tools, orchestration, and runtime mechanisms.

3 Experimentation and Validation

Systematically test and compare models, prompts, contexts, tools, and system configurations to validate performance and guide design decisions.

4 Observability and Continuous Improvement

Monitor real-world behavior, identify limitations and failure modes, and use evidence to guide ongoing improvements.

CONTINUOUS IMPROVEMENT

An Interconnected Improvement System

Systemic perspective	Improving GenAI performance requires coordinated interventions across the model, harness, and system configuration.
Evidence-driven decisions	Experiments and production observability reveal what works, what fails, and what needs improvement.
Iterative evolution	Feedback loops connect evaluation, adaptation, and deployment, enabling GenAI systems to continuously improve over time.

KEY TAKEAWAY

Improving GenAI systems is not about optimizing the model alone. It is about continuously improving the entire system.

Why Model Optimization Alone Is Not Enough

Model optimization can improve capabilities, but it cannot address all the limitations of GenAI systems.

Cost and complexity	Fine-tuning and other optimization techniques can require substantial computational resources, specialized expertise, high-quality data, and ongoing maintenance.
Knowledge limitations	Model optimization alone does not provide reliable access to up-to-date, proprietary, or task-specific contextual information.
Memory limitations	It does not inherently provide persistent memory, long-term state management, or overcome context-window constraints.
Capability limitations	Models still need external tools to retrieve information, perform precise computations, interact with systems, and execute actions.
Reliability limitations	Optimization does not eliminate hallucinations, reasoning errors, or the need for runtime controls and validation.

KEY TAKEAWAY

Model optimization improves what a model can do; the harness extends what the system can access, remember, control, and execute.

Training models is expensive; improving the system is not

WHERE YOU IMPROVE	TYPICAL COST (A SENSE OF SCALE)	WHO DOES IT	TIME TO CHANGE
Pre-training	**$$$$$** $10M–$100M+ per run; e.g. Grok 4 ≈ $490M	Frontier labs	🕒🕒🕒🕒 Months; produces a new model
Post-training	**$$**$$$ ~$1M–$5M, depending on the technique. Frontier labs don't disclose; DeepSeek-R1's RL stage ≈ $0.3M	Labs, enterprises	🕒🕒🕒 Weeks; redone when the base model changes
Harness & context	**$**$$$$ Engineering time + pay-per-use inference	Product teams	🕒 Days; versioned and reversible
Experiments & observability	**$**$$$$ Tooling + engineering time	Product teams	🕒 Continuous

Same models for everyone

Most companies use the same frontier models through an API. The advantage lies in the system built around them.

Cheap, fast, reversible

Harness changes ship in days, cost little, and carry over when a better model arrives.

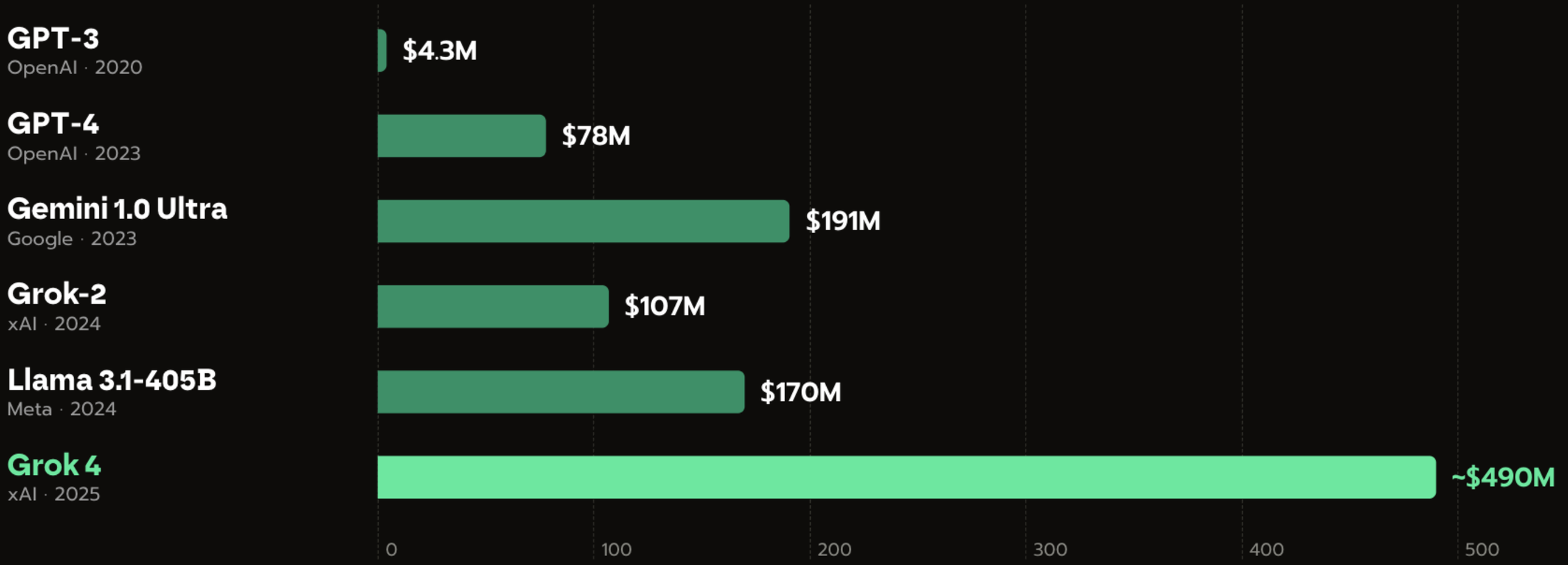
Evidence before spend

Experiments and observability show where the system fails, so effort goes where it pays off.

Indicative ranges, to give a sense of scale. Sources: Stanford HAI, AI Index 2024 (Epoch AI estimates); Epoch AI, Grok 4 (2025); DeepSeek-R1, *Nature* (2025).

The rising cost of training frontier models

Estimated compute cost of the final training run, in millions of US dollars.



Third-party estimates of compute for the final training run, priced at cloud-rental rates; they exclude research, staff, data and failed runs. Frontier training costs have grown ~2.4× per year since 2016 (95% CI 2.0–3.1×; Cottier et al., Epoch AI, 2024).
Sources: Stanford HAI, AI Index Report 2024 (GPT-3, GPT-4, Gemini 1.0 Ultra) and 2025 (Grok-2, Llama 3.1-405B), both using Epoch AI estimates; Epoch AI, *What did it take to train Grok 4?* (Sept 2025, median estimate).

Main takeaways

Leave model training to the frontier labs

Building a new foundation model, or training one further, is frontier-lab work that costs tens to hundreds of millions. And training alone can't fix knowledge cut-offs, memory, actions or hallucinations.

Grok 4 ≈ $490M per training run.
OpenAI: accuracy will never reach 100%

The harness can matter more than the model

Context, memory, tools, orchestration and guardrails expand and control what the model can do. The same model performs very differently in a different harness.

GPT-5 on Terminal-Bench 2.0: 33.9% → 49.6%, changing only the harness

Improve continuously, with evidence

Building and maintaining a GenAI system is a loop. Invest in experiments and observability, and let feedback, data and facts drive each change.

Read real traces → find failure modes → fix → measure again

Sources: Epoch AI, Grok 4 training resources (2025); OpenAI, *Why language models hallucinate* (2025); Terminal-Bench 2.0 leaderboard, as reported in *A Survey on Agent System and Harness Design* (arXiv, 2026); H. Husain & S. Shankar, AI evals guidance (2025–26).

03. Context & **Context Engineering**

The model only knows what reaches its window. Engineer what goes in.

What context is | Context engineering | vs. prompt engineering | Building blocks

ONE IDEA TO KEEP

A model can only be as good as what it sees. Context is where your data, rules and know-how enter the system.

Engineer the window, not just the prompt.

LESSON

Context

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

Understand what context is: everything the model sees when it answers. Know the building blocks that fill it (instructions, knowledge, memory, tools and their results), and what context engineering is: designing the system that selects, structures and delivers the right information at each step.

What is context?

Everything the model sees when it generates a response: the tokens in its context window.

Weights + context are all it knows

At inference, the model can only use what it learned in training and what is in the window.

The window is finite

Every token competes for space and for the model's attention.

More is not better

Accuracy drops as inputs grow longer ("context rot"), so relevance beats volume.

Sources: Anthropic, *Effective context engineering for AI agents* (2025); Chroma Research, *Context Rot* (2025).

What is context engineering?

Designing the systems that select, structure, and deliver the right information to the model at each step.

"The delicate art and science of filling the context window with just the right information for the next step."

Andrej Karpathy (2025)

"Curating and maintaining the optimal set of tokens during LLM inference."

Anthropic (2025)

Prompt engineering and context engineering, side by side

Prompt engineering

Writing and organising the **instructions** so the model responds well.

QUESTION IT ANSWERS

How should I **say** it?

UNIT OF WORK

One prompt: role, task, examples, format

WHEN IT HAPPENS

Written once, before the call

TYPICAL FAILURE

Vague or ambiguous wording

Context engineering

Curating **everything in the limited context window**: instructions, knowledge, memory, tools, history.

QUESTION IT ANSWERS

What should the model **see** right now?

UNIT OF WORK

The whole window, and the pipeline that fills it

WHEN IT HAPPENS

At runtime, every time the model is called

TYPICAL FAILURE

Missing, stale or too much information

Context engineering is the natural progression of prompt engineering: the prompt is still there, now one part of the window.

Source: Anthropic, *Effective context engineering for AI agents* (2025).

Context engineering vs. prompt engineering

Prompt engineering

What to say. Wording one instruction well.

Context engineering

What the model sees. A system that fills the window at every step.

	PROMPT ENGINEERING	CONTEXT ENGINEERING
Scope	One instruction	Everything in the window: instructions, knowledge, memory, tool results
When	Written once, before the call	Assembled at runtime, for every step of a task
Skills	Writing, examples, formatting	Retrieval, data and memory design, orchestration, evaluation

Prompt engineering is part of context engineering.

Context engineering in the harness

Open the interactive examples ↗

WHAT

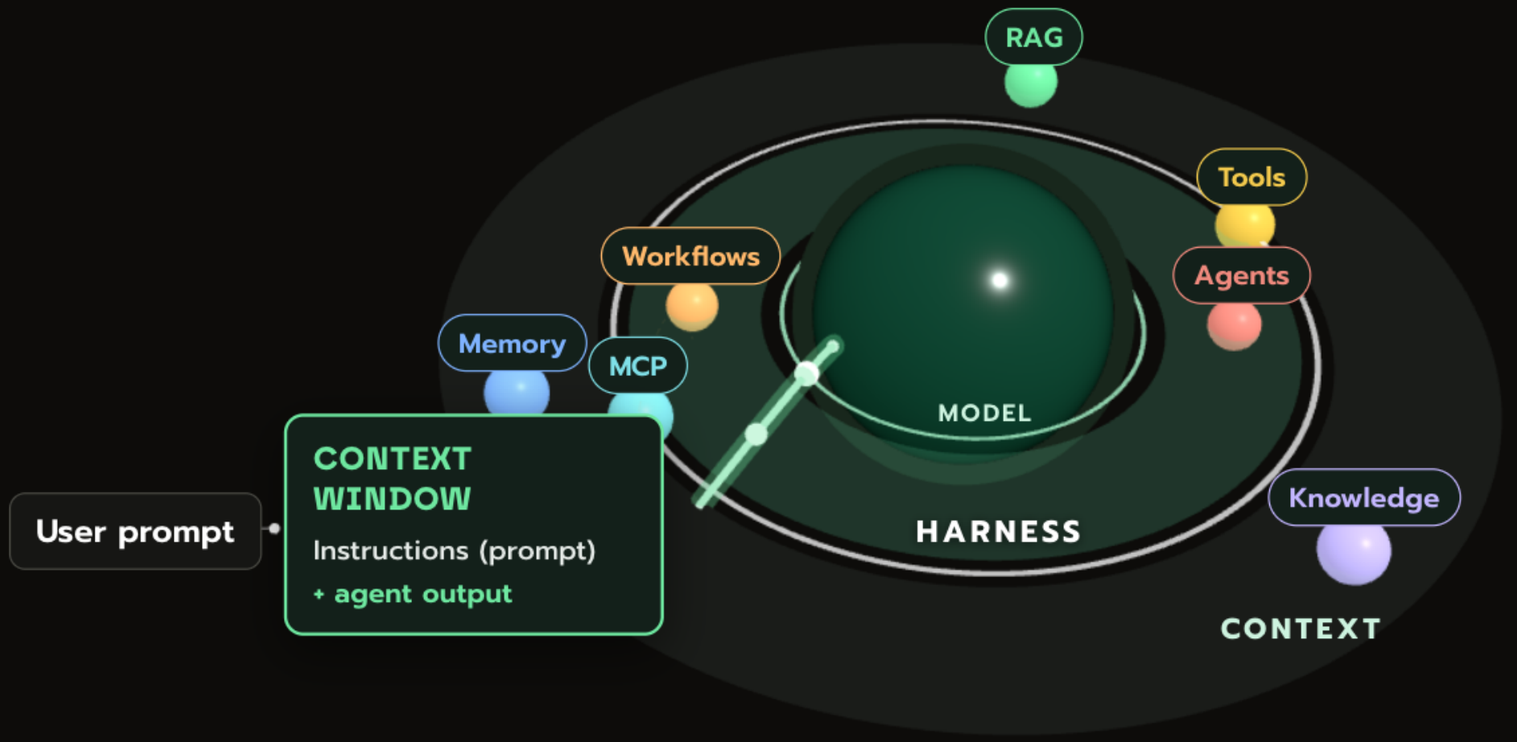
Design and develop the contextual knowledge and systems that support task execution.

HOW

Runtime mechanisms that dynamically retrieve, assemble, prioritize and inject relevant information into the context window.

WHY

Better performance and reliability, especially in specialized domains with specific and proprietary data.



CONTEXT · REFERENCE

Building blocks of context

The pieces a harness draws on to fill the context window, grouped by role.

Not exhaustive. A common list for instructional purposes; real systems combine, rename and extend these blocks.

Knowledge

Documents (Markdown)
Curated text the model can read directly.

Knowledge graphs
Entities and the relations between them.

Ontologies
Formal concepts, categories and rules of a domain.

Semantics
Shared meaning: glossaries, definitions, metric logic.

Schemas

Memory
What the system remembers, and in what shape.

Entity
The structure of domain objects (customer, order…).

Input
The expected format of requests and parameters.

Output
Structured responses the system can validate (e.g. JSON).

Prompts & skills

Prompts
Reusable instructions, roles and templates.

Skills
Packaged instructions, scripts and resources, loaded only when a task needs them.

Rules & policies

Business rules
Domain logic the system must follow.

Guardrails
Safety, compliance and quality checks on inputs and outputs.

Repositories

Databases
Structured records, queried for exact facts.

Vector stores
Embeddings that enable search by meaning.

Memory stores
Persisted conversation, user and task memory.

Retrievers (RAG)
Vector: semantic similarity. **Hybrid**: keywords plus vectors. **Graph**: follows relations between entities. **Agentic**: the model decides what to retrieve, and when.

Main takeaways

Your context is your competitive edge

Everyone can use the same frontier models. Only your organization has its data, processes, rules and know-how. The better that context is curated, structured and kept current, the better the outcomes.

Top foundation models now score within a few points of each other on common benchmarks

Context engineering builds runtime infrastructure

Its outputs (knowledge bases, schemas, prompts and skills, rules, retrievers and memory) are assets the harness draws on at every step, giving the model the right information, in the right format, at the right time.

"Building dynamic systems to provide the right information and tools in the right format." H. Chase, LangChain

Sources: Contrary Research, *The case for specialized AI* (2025); Gradient Flow, *Your AI playbook for the rest of 2025*; H. Chase, *The rise of "context engineering"*, LangChain (June 2025).

The right information, at the right time

WHAT A MODEL CAN'T KNOW ON ITS OWN

It knows only its training and what is in its context window **right now**.

Your private data
never in its training set

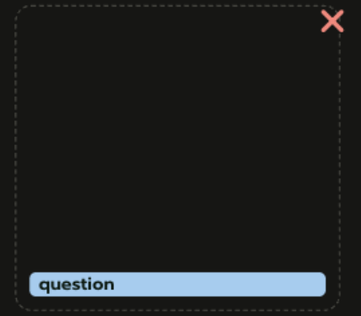
Today's facts
after its training cut-off

This customer
history, preferences, open issues

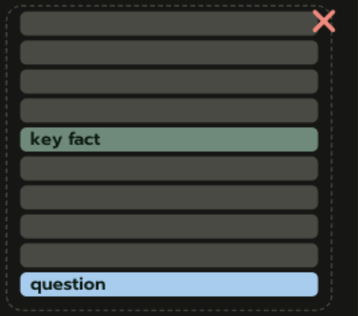
The task's current state
steps done, results so far

Context engineering brings these in.

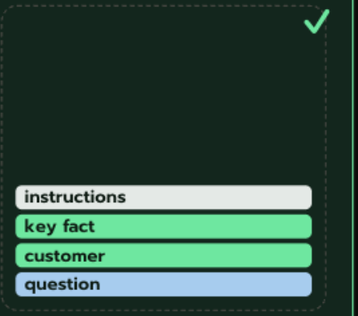
HOW MUCH TO GIVE IT



Too little
It fills the gaps with guesses.



Too much
The key fact gets lost in the middle; accuracy drops.



Just right, just in time
Small, relevant, current: the answer is right.

Context engineering gives the model what training never could, and only the portion this step needs.

Sources: Anthropic, *Effective context engineering for AI agents* (2025); Liu et al., *Lost in the Middle*, TACL (2024); Chroma Research, *Context Rot* (2025).

LESSON

RAG overview

DEPTH

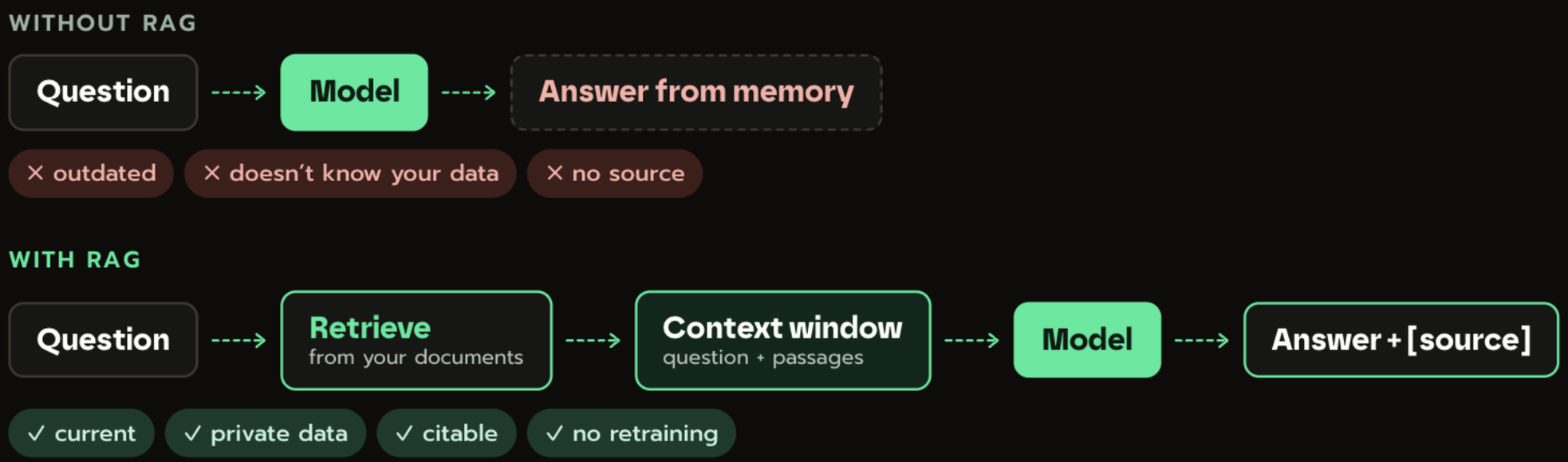
- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

A gentle introduction to Retrieval-Augmented Generation. Instead of retraining the model, the system finds the passages of your documents closest in meaning to the question and places them in the context window, so answers are grounded, current and citable.

What is RAG, and why?

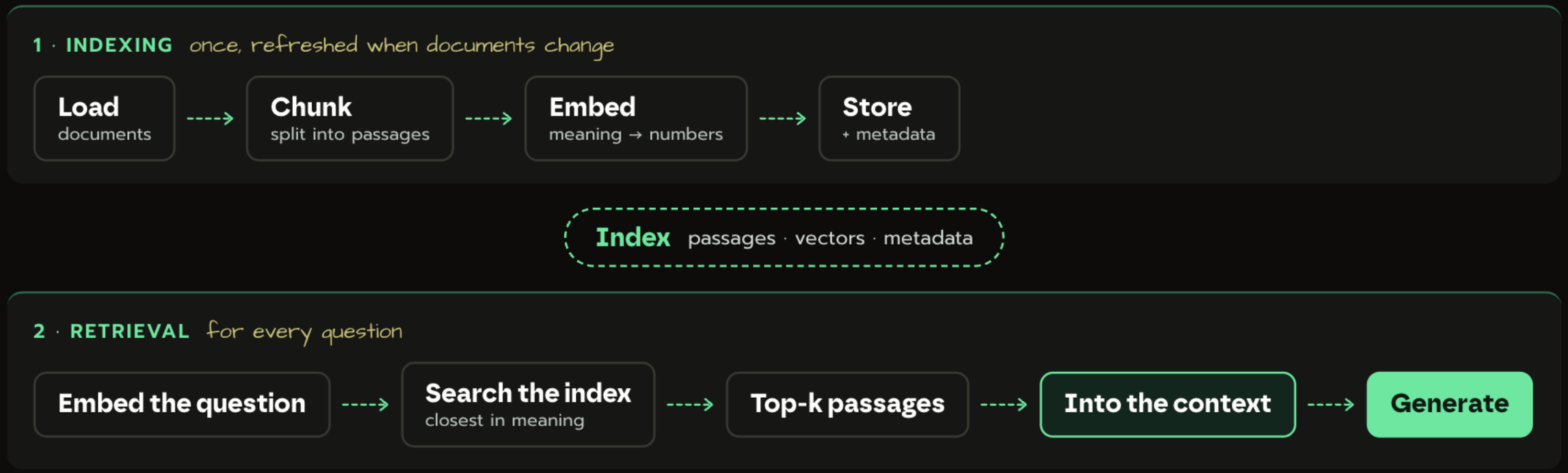
Retrieval-Augmented Generation: look it up first, then answer from what was found.



The model doesn't change. What changes is what it sees: the right passages from your documents, placed in the context window at question time.

Source: Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS (2020).

Two phases: indexing and retrieval



Indexing decides what can be found; retrieval decides what the model sees.

Indexing: every step is a design choice

1
Load
Get the content in

- PDF & web parsing
- Databases, APIs
- OCR for scans & images

2
Chunk
Split into passages

- Fixed size + overlap
- By headings / structure
- By meaning

3
Enrich
Add what helps find it

- Metadata: source, date, access
- Summaries
- Context per chunk

4
Embed
Turn meaning into numbers

- Text models
- Multimodal (text + images)
- Domain-tuned models

5
Store
Make it searchable

- Vector database
- Keyword index
- Both (hybrid)

Not exhaustive: common options, not a complete list. The best mix depends on your documents and questions, so measure it.

Source: Anthropic, *Contextual Retrieval* (2024).

Retrieval: every step is a design choice

1
Rewrite
Improve the question

- Rephrase
- Expand with synonyms
- Split into sub-questions

2
Search
Find candidates

- Semantic (vectors)
- Keyword
- Hybrid: both

3
Filter
Keep only valid ones

- Metadata: date, product
- Permissions

4
Rerank
Best ones first

- Reranker model
- Recency, source quality

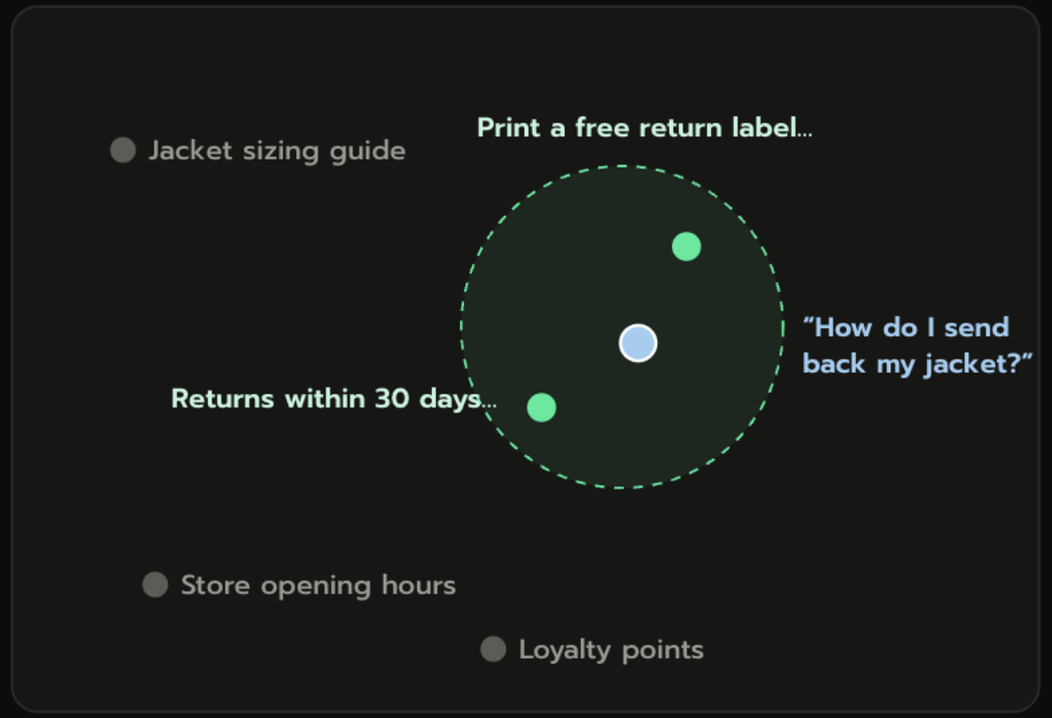
5
Assemble
Build the context

- Top-k, sensible order
- Cite sources
- "Say if not found"

Not exhaustive: common options, not a complete list. Each step trades quality against cost and speed.

Source: Anthropic, *Contextual Retrieval* (2024): hybrid search + reranking cut failed retrievals by ~⅔.

Semantic search: finding by meaning



KEYWORD SEARCH

"send back" → 0 results

No article contains those words.

SEMANTIC SEARCH

"send back" → "return label"

Different words, same meaning: the passages sit close together in the embedding space.

Closeness is measured with **cosine similarity**: how much two meanings point the same way.

Metadata and hybrid search: precision and permissions

METADATA FILTERS

The right version

Date, product, language, region. “Only the 2026 returns policy, Portugal.”

ACCESS CONTROL

Only what you may see

A store clerk never retrieves HR files, even if they match the question.

HYBRID SEARCH

Meaning + exact words

Codes, names and IDs that embeddings blur: order `#4512`, error `TS-999`.

Meaning finds the candidates; metadata makes them the right ones.

RAG in action

An instructional simulation: a real RAG pipeline, run one visible step at a time.

Open the app ↗

Load 10 Markdown documents ----> **Chunk** 6 strategies to compare ----> **Enrich** metadata on each chunk ----> **Embed** local MiniLM or OpenAI ----> **Store** a FAISS index ----> **Retrieve & chat** dense, keyword or hybrid ----> **Analytics** topics in support questions

WHAT YOU SEE

The real data at each step

Chunks, metadata, vectors and scores, and exactly what reached the model.

WHAT YOU CHANGE

One choice at a time

Chunking, embedder, retrieval mode, top-k. Then compare the answers.

THE DATA

A company that doesn't exist

Nimbus Logistics, an invented courier. Its facts are fictional, so a correct answer must come from retrieval.

Run it from the repo root: `uv run rag-in-action`. The chat needs an OpenAI key; every other step works without one.

Beyond basic RAG: other types

Same idea, look it up before answering, with smarter ways to find and check.

AGENTIC RAG

An agent decides

It plans, picks sources, searches again if the first results aren't enough.

"Compare our 2025 and 2026 return policies" → two searches, then a comparison.

GRAPH RAG

Facts that connect

Retrieves from a knowledge graph: entities and their relations, not just passages.

"Is my jacket in the recall?" → order → batch → supplier → recall.

MULTIMODAL RAG

Beyond text

Indexes images, tables, audio and video alongside text.

A photo of a damaged zip finds the matching repair guide.

CORRECTIVE / SELF-RAG

It checks itself

Grades what it retrieved; if it's weak, it searches elsewhere or says so.

No good passage found → falls back to the web, or answers "I don't know".

STRUCTURED RAG

Ask the database

Turns the question into a query (e.g. SQL) over tables.

"How many returns in October?" → a query, not a passage.

Not exhaustive: the most prominent variants, and they are often combined.

Sources: Asai et al., *Self-RAG* (2023); Yan et al., *Corrective RAG* (2024); Edge et al., *From Local to Global: A Graph RAG Approach*, Microsoft (2024).

Main takeaways

1. **RAG gives the model your knowledge without retraining it.**
   Current, private and citable, at question time.

2. **The answer is only as good as what is retrieved.**
   Most RAG quality is decided in indexing and retrieval, not in the model.

3. **Every step is a design choice: measure it.**
   Chunking, search, filters and permissions should be tested on your real questions.

04. Tools and **Integration**

The model decides what to call. Your code decides whether and how it runs.

Why models need tools | The tool loop | Five examples | MCP | Least privilege

ONE IDEA TO KEEP

Connecting a model to tools does not make it smarter. It changes its reach, and with it, what can go wrong.

Same weights, new abilities, new risks.

A trained model has four gaps

It is a frozen set of weights that predicts the next token. Tools fill what that design leaves out.

GAP

Fresh information

Training stopped at a cutoff date.

Weather API, web search

GAP

Reliable computation

It predicts digits instead of calculating them.

Calculator, code

GAP

Private data

Your data was never in its training set.

Database, internal API

GAP

The ability to act

Text alone changes nothing in the world.

Create ticket, send email

The tool loop

A tool is a function in your program, described to the model by a name, a purpose and its arguments.

1 · Your code
sends the question and the tool descriptions

---->

2 · Model
replies with a request:
`get_temperature(city="Lisbon")`

---->

3 · Your code
runs the function and sends back the result: `19.4 °C`

---->

4 · Model
answers in text: "It is 19.4 °C in Lisbon."

Steps 2-3 repeat until the model replies with text instead of a tool call. Agent frameworks add features around this loop; the core is these few lines.

The model never runs anything. A tool call is structured output. Your code executes it, so control lives in that gap.

The model only reads the description

```
# What the model sees: never the Python code
{
  "name": "get_temperature",
  "description": "Current air temperature
     in °C for a city. Use for questions
     about today's weather.",
  "parameters": {
    "city": { "type": "string" }
  }
}
```

1. **Choice comes from text.** The model picks a tool and fills its arguments from the name and description alone.
2. **Nothing maps the question to the function.** The model extracted `city="Lisbon"` itself.
3. **Write descriptions with care.** They guide the model; they don't guarantee what it does.

Five examples, one loop

Open the interactive notes ↗

FRESH INFORMATION

1 · Temperature

Without the tool it can only guess a typical figure. With it, it calls the weather API.

CHAINED

2 · Temperature + activities

The first result becomes the second tool's input. The model plans the order.

OPEN-ENDED

3 · Search

The model writes its own query. The answer is grounded, not remembered.

COMPUTATION

4 · Calculator

Exact arithmetic instead of predicted digits, checked against the formula.

PRIVATE DATA · ACTION

5 · Your systems

Look up data no model was trained on, and change something in the world.

RESTRAINT

When not to use a tool

General knowledge needs no call. Every call costs time and tokens.

TOOLS · HANDS-ON

How LLMs use tools

Open the interactive notes ↗

`OPENAI_API_KEY` `TAVILY_API_KEY for example 2` `~70 cells`

The whole mechanism

The loop below is all there is. The model returns either text or a request to call a tool; your code runs the tool and sends the result back.

```
messages = [{"role": "user", "content": question}]
while True:
    reply = model(messages, tools=TOOL_DESCRIPTIONS)
    if not reply.tool_calls:
        return reply.text                    # done: an answer in text
    for call in reply.tool_calls:
        result = TOOLS[call.name](**call.arguments)   # your code runs it
        messages.append(tool_result(call.id, result))
```

The five examples

1. Temperature: fresh information

WITHOUT TOOL

The model can't know today's value. It declines, or offers a typical figure for the season.

WITH TOOL

It calls `get_temperature` with `city="Lisbon"`. Our code calls Open-Meteo; the model only turns the JSON into a sentence.

2. Temperature + activities: chained tools

The second tool needs the first tool's result. Our code never connects them: the model passes `temperature_c` from one call to the next because the description says it needs it.

3. Search: open-ended

Asked about an event after its training cutoff, a model without tools may even insist it hasn't happened yet: its sense of "now" is frozen. With search it writes its own query and answers from the article. The answer is only as good as the source.

4. Calculator: reliable computation

The tool computes; the model only sets up the call. The notebook checks the result against the formula. Note that `calculate` does not use `eval`: arguments come from the model and are untrusted.

5. Private data and actions

The model looks up an order that exists only in our (fake) data, and can take an action. This is where permission checks matter most.

TRY IT

1. Run Example 1 for your own city. Read the trace: which arguments did the model choose?
2. Rewrite a tool description to be vague. Does the model still call it, or call it correctly?
3. Ask a question none of the tools covers. Does the model call a tool anyway?

Takeaways

1. **The model requests, your code executes.** A tool call is structured output, not execution.
2. **The loop is small.** Send the question and tools, run what is asked, send results back, repeat.
3. **Descriptions are prompts.** The model chooses tools from their names and descriptions.
4. **Tool arguments are untrusted input.** Validate them as you would any external request.
5. **Tool results are untrusted too.** A web page could contain text that tries to instruct the model.
6. **Separate reads from writes.** For tools that change state, add permission checks or human confirmation in code, not only in the description.

Tools give reach. Code keeps control.

1. **The model requests, your code executes.**
2. **The loop is small:** ask, run, return, repeat.
3. **Descriptions are prompts.** Write them with care.
4. **Arguments are untrusted input.** Validate them.
5. **Results are untrusted too.** They can carry instructions.
6. **Separate reads from writes.** Check permissions in code.

LESSON

MCP overview

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

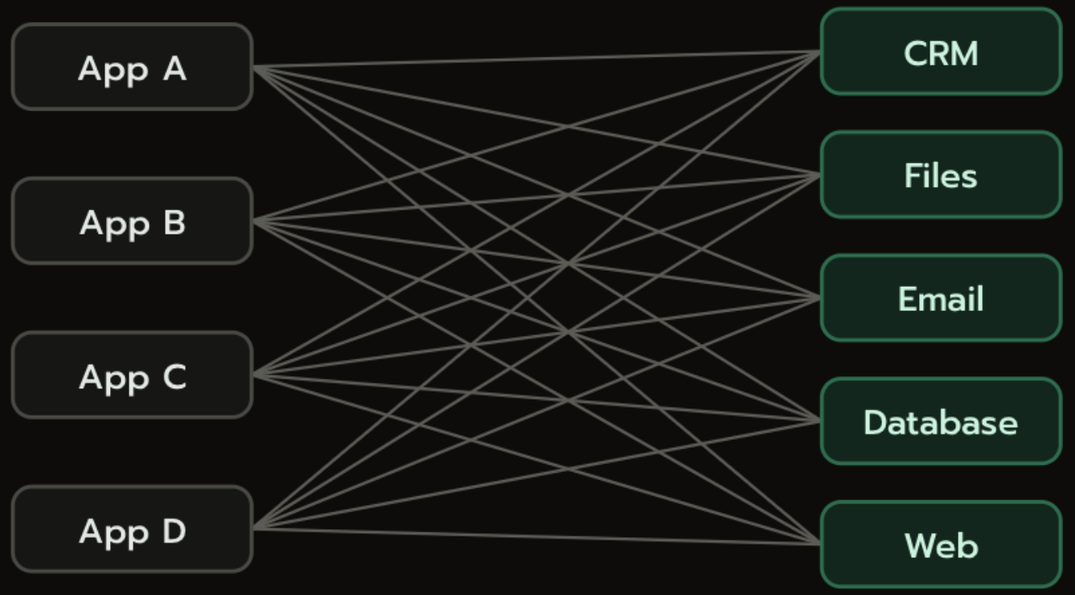
OBJECTIVE

MCP: one plug, many systems. The Model Context Protocol standardises where tools come from and who runs them. Understand how a host connects to a server, discovers what it offers, and uses it safely.

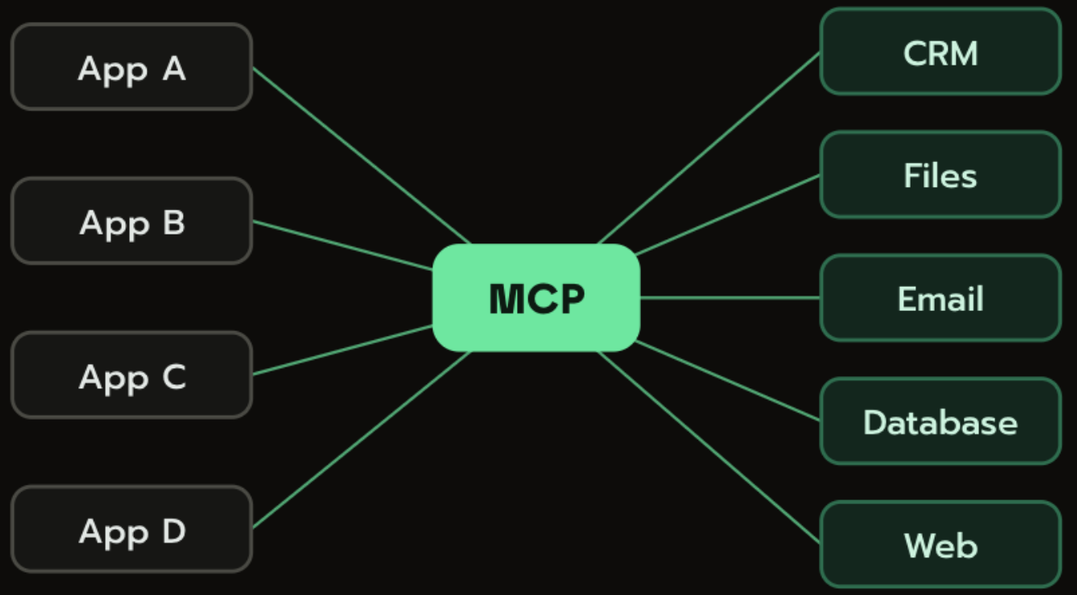
From N × M integrations to N + M

An open standard introduced by Anthropic in 2024, often called "USB-C for AI".

Without a standard · 4 apps × 5 systems = **20** connectors



With MCP · 4 clients + 5 servers = **9**



The MCP client and the MCP server

Two small programs that speak the same protocol. The client lives in your app; the server wraps a system.

INSIDE THE HOST · YOUR APP

MCP client

1. **Connects** to one server and does the handshake
2. **Translates** its tools into function calls the model understands
3. **Runs the loop:** the model asks, the client calls, the result goes back

Model · requests, never runs

JSON-RPC messages

`initialize` · hello, which protocol?

`capabilities` · tools, prompts…

`tools/list`

`fetch(url) + description`

`tools/call` · fetch(url=…)

`result` · page text

A SEPARATE PROGRAM

MCP server

1. **Describes itself:** its tools, resources and prompts
2. **Runs each call** on its own system
3. **Enforces its limits:** a folder, an account, read-only

→ the web · your files · a CRM

Local server
The client starts it from a command line, e.g. `uvx mcp-server-fetch`, and talks over stdin/stdout (stdio).

Remote server
It runs elsewhere, as a web service. The client connects over Streamable HTTP.

Same client code, different server: in the notebook, only the launch command changes.

The moving parts

Open the interactive notes ↗

ROLE	WHAT IT IS
Host	The app the user works in: a chat app, an IDE, a notebook
Client	Inside the host; holds one connection to one server
Server	A program exposing capabilities: fetch a URL, read files, query a CRM

A SERVER OFFERS	WHO DECIDES TO USE IT
Tools	The model · `fetch(url)`
Resources	The application · a file or schema to attach as context
Prompts	The user · a reusable template, like a slash command

MCP does not replace function calling. The model still makes ordinary tool calls. MCP standardises where the functions come from, and the host still runs them.

Ask the server what it can do

Discovery is part of the protocol: a server must describe itself. No guessing, no trusting the README.

`initialize`

Who are you?

Name, version and the capabilities it declares.

`tools/list`

What can the model do?

Each tool's name, description and input schema.

`resources/list`

What data can I attach?

Files, records, schemas the app can read.

`prompts/list`

Which templates?

Reusable prompts the user can pick.

Declared does not mean available. Run the list query and handle error `-32601` ("method not found"). The model's own summary of its tools can be wrong; `tools/list` is the source of truth.

Least privilege, in three layers

A server that can read your files can often write them too. Give the model only what the task needs.

LAYER 1 · SERVER

Scope it

The server enforces its allowed folder or account, whatever the client asks.

LAYER 2 · CLIENT

Filter the tools

Your code chooses which tools the model sees. The layer you control.

LAYER 3 · ANNOTATIONS

Read the hints

Labels like `read_only` help you choose. Trust them only from trusted servers.

Read-only tools offered

"Create notes.txt" → **disk unchanged**

All tools offered

"Create notes.txt" → **file was created**

Enforced by code, not by asking the model to behave: a tool it cannot see cannot be misused.

When to use it, and when not to

USE MCP WHEN

1. Several apps or agents need **the same integration**
2. You want **existing servers** instead of writing connectors
3. Tools should be **discoverable at runtime**

SKIP IT WHEN

1. One script needs one function: **plain function calling** is simpler
2. A **fixed API call** is enough and no model has to choose
3. Each extra tool schema adds **cost you can't justify**

Before you connect a new server

It runs with your permissions, and the model will read whatever it returns.

1

Trust

Do you trust who wrote it?

2

Discover

List its tools, resources and prompts. Don’t trust the README.

3

Restrict

Offer the model only the tools it needs. Scope folders and accounts.

4

Test

Try it with the MCP Inspector before a model uses it.

One plug, many systems. Your code keeps control.

1. **MCP changes reach, not intelligence.** Same weights, new abilities.
2. **A standard, not a tool.** One client, any server.
3. **The model requests, the host executes.**
4. **Verify outside the model.** Check facts and disk, not claims.
5. **Grant the least privilege.** Scope servers, filter tools.
6. **Every connection has a price:** tokens, time, prompt injection.

⚠ SECURITY ALERT · USING THIRD-PARTY TOOLS

Third-party tools and MCP servers run with your permissions

RISK	WHAT CAN HAPPEN	WHAT TO DO
Hidden instructions	A tool's description can carry orders the model follows and users never see ("tool poisoning").	Use vetted sources only. Review what each tool tells the model.
Poisoned content	Web pages, emails and files returned by a tool can instruct the model to leak data or act.	Treat every tool result as untrusted. Require human approval before actions.
Fake or changing packages	A look-alike "postmark-mcp" package copied users' emails to an attacker (2025). Approved servers can change after install.	Install from the vendor's official source. Pin versions; re-review on every update.
Too much access	A server that can read your files or mailbox can often change or send them too.	Least privilege: scoped accounts, read-only by default, only the tools the task needs.
Vulnerable connectors	A flaw in *mcp-remote* let a malicious server run code on the user's computer (CVE-2025-6514, severity 9.6/10).	Keep an inventory, patch fast, connect only to trusted servers over HTTPS.

Sources: OWASP Top 10 for LLM Applications 2025 (LLM01 Prompt injection, LLM06 Excessive agency); Invariant Labs, *tool poisoning* (Apr 2025); postmark-mcp incident reports (Sep 2025); JFrog, CVE-2025-6514 (Jul 2025).

⚠ SECURITY ALERT · BUILDING YOUR OWN TOOLS

Building tools and MCP servers: security is part of the design

PRACTICE	WHY IT MATTERS	WHAT TO REQUIRE
Validate every input	Arguments are written by a model, and the model can be steered by an attacker.	Strict schemas and allow-lists. Never pass arguments straight to code, a shell or a database query.
Authenticate every call	The server must know who is asking and what they may do, not trust the model's word.	Scoped, short-lived credentials. Never accept or forward tokens not issued to your server.
Least privilege by design	One broad tool is one big risk. Reads and writes carry very different stakes.	Small, focused tools. Confirmation or approval for anything destructive or costly, enforced in code.
Honest descriptions	The description is a prompt: it steers the model, and other people's agents will read it.	Say exactly what the tool does. Label read-only and destructive tools. No secrets in error messages.
Limit, log and test	A model can call a tool thousands of times, and every call must be traceable afterwards.	Rate limits and budgets, audit logs, dependency scans and security testing before release.

Sources: Model Context Protocol specification, *Security Best Practices* (2025-11-25); OWASP Top 10 for LLM Applications 2025 (LLM01 Prompt injection, LLM05 Improper output handling, LLM06 Excessive agency).

05. Orchestrators: **Workflows and Agents**

Who decides the next step: your code or the model?

Prompt → model | Workflows | Agents | When to use which | Agentic systems | Agentic RAG

ONE IDEA TO KEEP

A workflow follows the path you drew. An agent chooses its own path.

Both are orchestration. Only the decider changes.

LESSON

Overview: workflows and agents

DEPTH

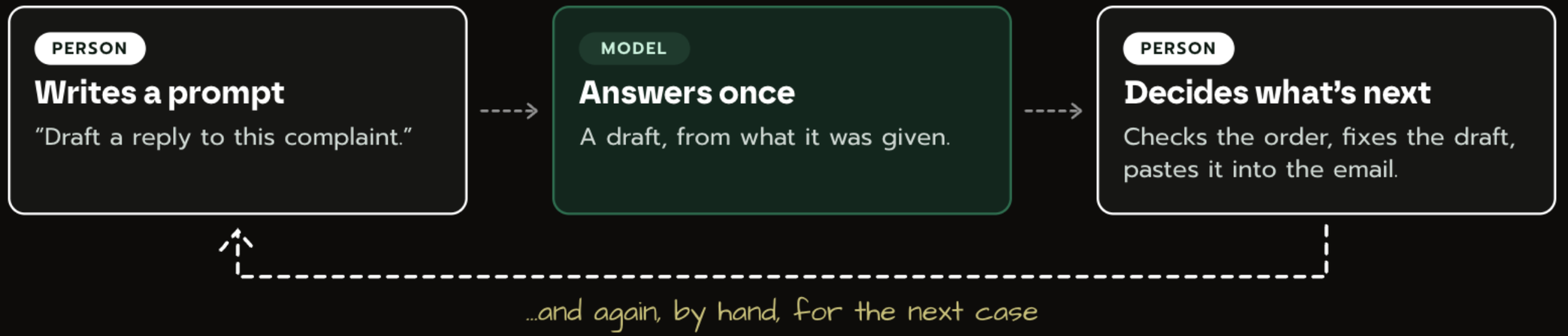
- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

Understand the orchestrator: the part of a GenAI system that decides the next step. Tell apart a single prompt, a workflow (your code decides) and an agent (the model decides), and know when each one fits.

Prompt → model: the person is the orchestrator

One question, one answer. Every next step is decided, and done, by hand.



Simple, but it doesn't scale. Nothing is automated or repeatable: the quality depends on who writes the prompt and what they do with the answer.

○ Person ▢ Model

The orchestrator decides the next step

Take the person out of the loop and something else must decide. There are two answers.

WORKFLOW

Your code decides the path

The steps are drawn in advance. The model fills in each step; the order never changes.

AGENT

The model decides the path

It gets a goal and tools, then chooses the next step, again and again, until it is done.

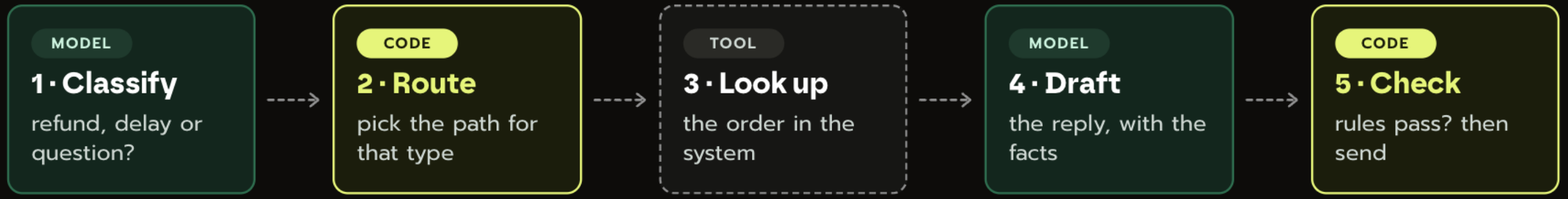
Orchestrator: the part of the system that decides what happens next.

"Workflows are systems where LLMs and tools are orchestrated through predefined code paths. Agents [...] are systems where LLMs dynamically direct their own processes and tool usage." Anthropic, *Building effective agents* (2024).

Model Workflow Agent Tool

A workflow: the same path, every time

Example: a customer-support request. The model does the language; your code fixes the order.



FIVE COMMON PATTERNS

Chaining
One step's output feeds the next.

Routing
Send each case to its own path.

Parallel
Split the work, run side by side, merge.

Orchestrator-workers
A lead model splits the job; workers do the parts.

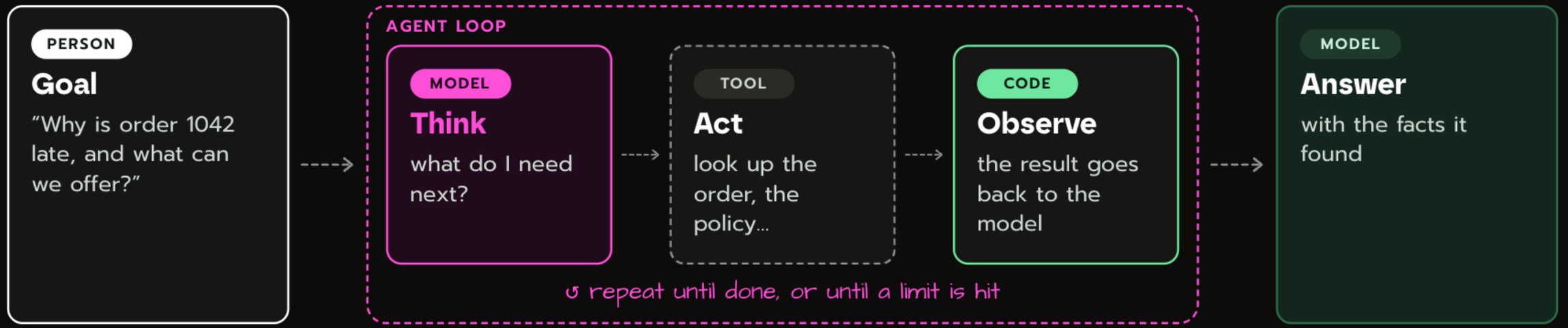
Evaluator-optimizer
One drafts, another critiques; repeat.

Patterns from Anthropic, *Building effective agents* (2024).

Model · Workflow step (code) · Tool

An agent: a model using tools in a loop

You give it a goal and a set of tools. It decides the steps, one at a time.



FLEXIBLE

Picks its own steps

Different cases, different paths. Nobody drew them in advance.

GROUNDED

Learns from results

Each tool result is "ground truth" that shapes the next step.

BOUNDED

Stops on purpose

When the goal is met, or at a step, time or cost limit set in code.

Anthropic, *Building effective agents* (2024): agents are "LLMs using tools based on environmental feedback in a loop".

Person · Agent (model decides) · Tool · Your code · Model

Three ways to use a model, side by side

	Prompt → model	Workflow	Agent
Who decides the next step	A person, every time	Your code, in advance	The model, while it runs
The path	One shot	Fixed and repeatable	Changes case by case
Predictability	High, but manual	High	Lower
Cost and speed	Lowest	Low to medium	Higher: more calls, more time
Best for	Single answers, drafts	Known, repeatable processes	Open-ended tasks, unknown steps
Main risk	Doesn't scale	Breaks on cases nobody foresaw	Wrong turns, loops, cost overruns

When to use which

Is one good answer enough, with a person checking it?	**Prompt → model** — a draft, a summary, an idea
Can you draw the steps in advance, and do they repeat?	**Workflow** — invoices, triage, standard replies
Do the next steps depend on what it finds along the way?	**Agent** — research, investigation, coding

Start simple. Use the simplest option that works, and add autonomy only when it pays for itself in results you can measure.

They mix: a workflow can contain an agent step, and an agent can call a workflow as a tool.

Prompt → model Workflow Agent

LESSON

Agents

DEPTH

Overview

Conceptual

Technical

Deep dive

OBJECTIVE

Tell a chatbot, an agent and an agentic system apart. See the agent as one component inside the harness, the main types of agents, and the common ways several agents are put together, including agentic RAG.

Chatbot, agent, agentic system

CHATBOT

It talks

Answers in a conversation. Behind it: a fixed script, a model, or both. It doesn't pursue a goal on its own.

scripted flow · LLM · chat memory

AGENT

It acts toward a goal

Plans, uses tools, checks the results and keeps going over many steps, without a person driving each one.

goal · tools · loop

AGENTIC SYSTEM

A team with rules

Several agents and workflows working as one system, with shared memory, tools and controls around them.

many agents · workflows · guardrails & evals

Note: "agent" is often a sales label. Ask what it can do on its own, and what it is allowed to change.

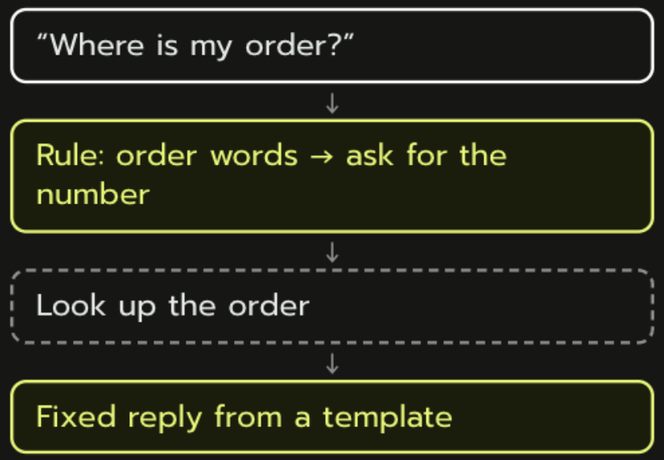
Agent · Workflow

A chatbot can follow a script, a model, or both

Many chatbots have no AI behind them: a step-by-step flow, drawn in advance.

SCRIPTED FLOW

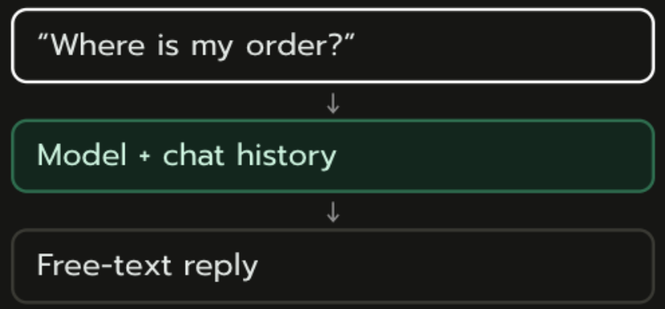
A decision tree



Code decides every step: same input, same answer. Predictable, auditable and cheap, but brittle off-script.

LLM CHATBOT

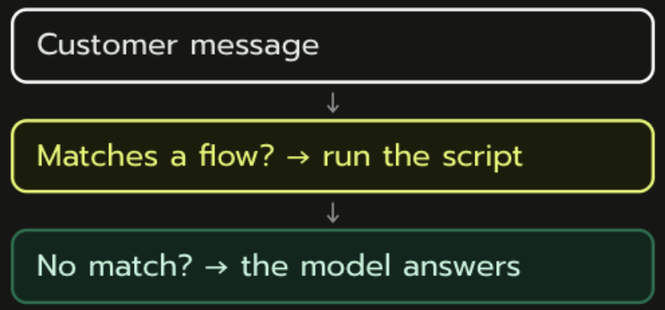
A model writes each reply



Understands any wording and sounds natural. With no tools it can't look anything up, and may invent.

HYBRID · THE MOST COMMON

Rules first, a model for the rest

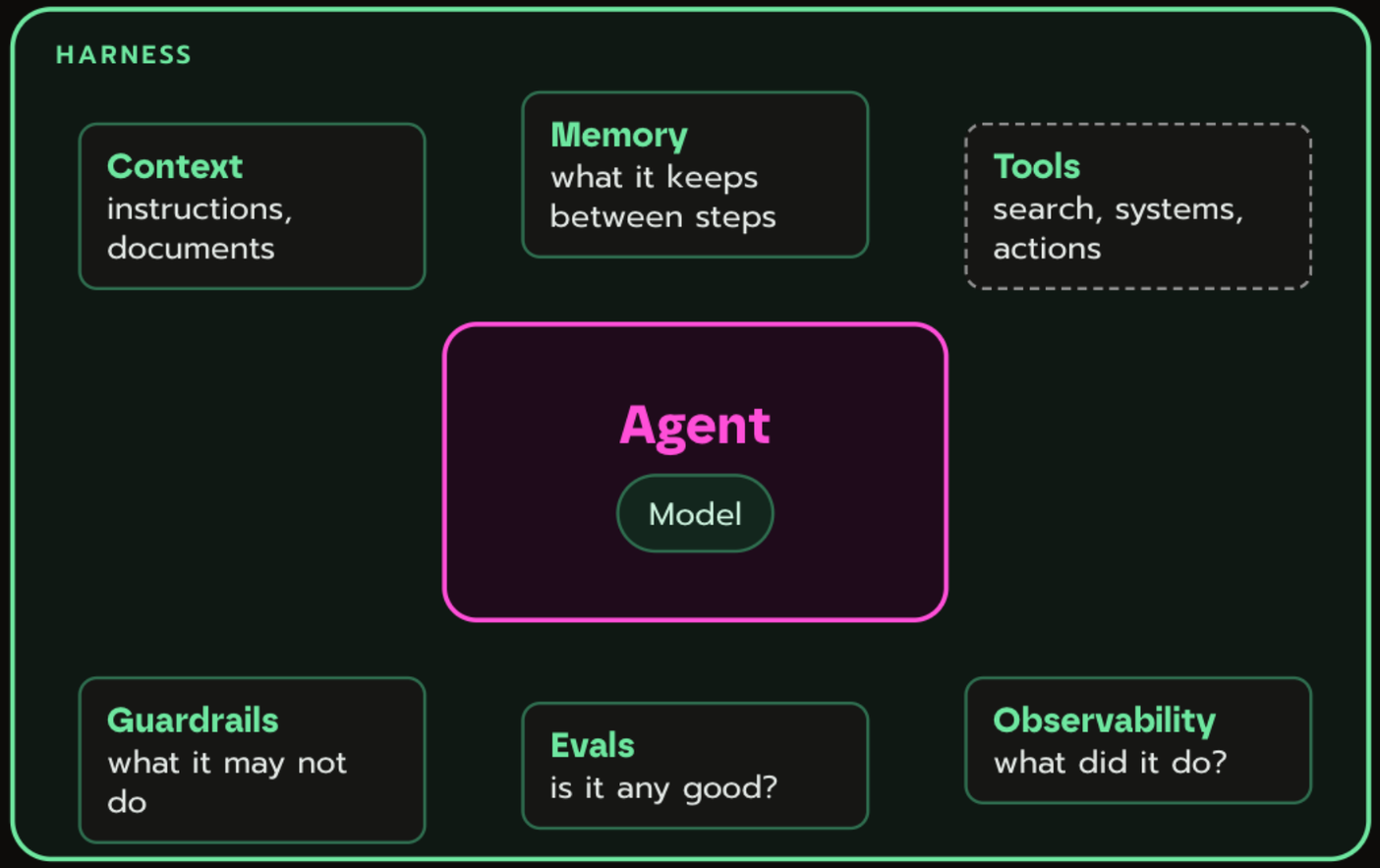


Control where it matters (orders, payments, identity); flexibility for open questions. The model is the fallback.

Google, Dialogflow CX docs: deterministic flows and "generative fallback" when input matches no intent. Microsoft, Copilot Studio docs: topics built from message, question and condition nodes.

Person · Script (code decides) · Model · Tool

The agent is one part of the harness



1. **The agent decides the steps.** The harness decides what it can see, use and change.
2. **Its quality comes from the parts around it:** the context it gets and the tools it can call.
3. **Same agent, different harness, different system.** The idea from Unit 1, one level up.

Harness (your code) · Agent · Model · Tool

Types of agents: by job and by autonomy

BY JOB

ASSISTANT

Helps a person

Answers, drafts and summarises inside the tools you already use.

e.g. Copilot, ChatGPT, Claude

TASK AGENT

Finishes a defined task

Uses tools to complete one job end to end.

e.g. process a refund, book a delivery

RESEARCH AGENT

Finds and compares

Searches, reads many sources and writes a cited report.

e.g. "deep research" features

CODING / COMPUTER USE

Works in software

Writes and runs code, or operates apps on a screen.

e.g. Claude Code, Codex, Cursor

BY AUTONOMY

more autonomy → more guardrails and evals

Suggests

A person reviews and takes every action.

Acts after approval

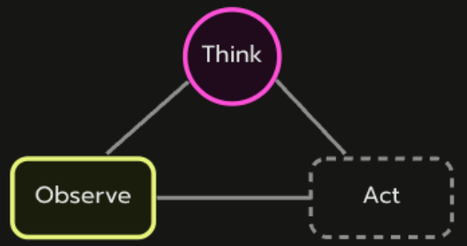
It prepares the action; a person says yes.

Acts on its own

Within limits set in code, and watched.

Agent

Four common ways to put agents together

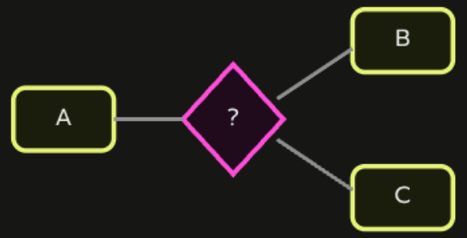


AGENT LOOP (REACT)

One agent, a few tools

Think, act, observe, repeat. The usual place to start.

A support agent that looks up orders and policies.

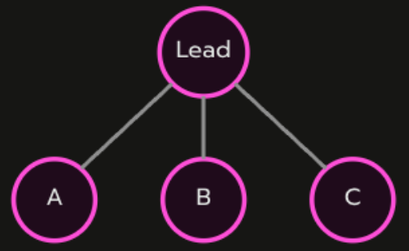


AGENT GRAPH

Steps drawn as a map

Code draws the boxes and arrows; the agent picks the branch. Easy to trace, pause and resume.

e.g. LangGraph

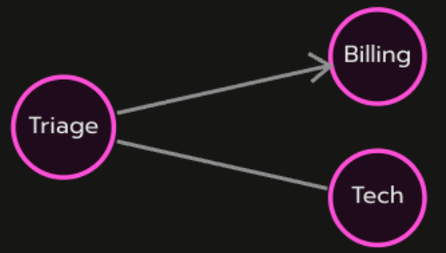


SUPERVISOR

A lead and specialists

A lead agent splits the job, specialists do the parts, the lead combines them.

Research: one searches, one checks numbers, one writes.



HANDOFF

Pass the case on

One agent hands the conversation to the specialist that fits, which takes over.

e.g. OpenAI Agents SDK handoffs

Real systems combine them, and often wrap them in a workflow.

Yao et al., *ReAct* (2022); Anthropic, *Building effective agents* (2024); LangGraph and OpenAI Agents SDK docs.

Agentic RAG: retrieval becomes a tool

The same RAG from Unit 3. What changes is who decides when, where and how often to look.

CLASSIC RAG · A WORKFLOW

Always one search

Question, then search, then answer. Same steps for every question.

AGENTIC RAG · AN AGENT

Searches as often as it needs

Decides whether to search, which source to use, and whether what it found is enough.

"Compare our 2025 and 2026 return policies" → two searches, then a comparison.

More flexible, and more to check. More searches cost more time and money. The answer is still only as good as what it retrieves.

Workflow Agent Tool Model

Workflows and agents in action

The same question, asked five ways. Watch who decides each step.

Open the app ↗

Chatbot
Scripted flow or LLM: switch and compare.

Prompt → model
One shot. No memory, no tools.

Workflow
Fixed path: classify, route, tool, draft, check.

Agent
The model picks tools in a loop.

Agentic system
A lead, two specialists and a reviewer.

WHAT YOU SEE

Every step, folded

Reasoning, routing, tool calls and their results, one click to open.

WHAT TO TRY

One question, five chats

Compare the steps, the tool calls, the time and the answer. Each chat has a "How it works" page with its diagram.

THE DATA

A shop that doesn't exist

Invented orders, policies and weather, so every fact must come from a tool.

Run it from the repo root: `uv run w-and-a`. A Tools page lets you run each tool. Live runs need an OpenAI key; without one, the scripted chatbot still works and presets replay recorded runs.

Workflow Agent

Main takeaways

1. **Each one has its place.**
   A prompt, a workflow or an agent: the right choice depends on the problem, the solution and the stage, from prototype to production.

2. **Agents matter, but context is the fuel.**
   An agent is only as good as the information and tools it is given.

3. **Start simple, then measure.**
   Add autonomy only when evals show it pays for itself. That is the next unit.

06. Guardrails and **Evals**

Guardrails keep it safe on every request. Evals prove that it works.

What and why | Types of guardrails | Types of evals | LLM as a judge | Accelerators | Takeaways

ONE IDEA TO KEEP

You can’t trust what you don’t check: guardrails on every request, evals on every change.

Safe by design, proven by evidence.

LESSON

Guardrails overview

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

A model can be wrong, manipulated or over-eager, and now it can act. Understand what guardrails are, where they sit around the model, the main types, and the ready-made tools that implement them.

Guardrails: checks in code around the model

Rules that run on every request, before, during and after the model, and decide what is allowed through.

WHY

Models are probabilistic

The same question can get a different, and sometimes wrong, answer.

An invented refund rule

WHY

Models can be manipulated

Users, documents and web pages can carry instructions.

"Ignore your rules..."

WHY

Models now act

With tools, a mistake is no longer just text.

Refunds, emails, deletions

WHY

Mistakes are costly

Data leaks, false promises and harmful content have legal and brand impact.

Privacy, regulation, trust

Asking the model to behave is not a control. Instructions in the prompt guide it; guardrails in code enforce the rule, whatever the model does.

Three places to check: input, actions, output

BEFORE THE MODEL

Input

What is allowed in.

- Validation: size and format
- Personal data redaction
- Prompt-injection detection
- Topic scope
- Harmful-content moderation

---->

WHILE IT ACTS

Execution

What it is allowed to do.

- Tool access by role
- Argument and policy checks
- Approval thresholds: a human in the loop
- Budgets: steps, time, cost

---->

BEFORE THE USER

Output

What is allowed out.

- Format and schema
- Secret and data-leak filter
- Groundedness: claims match sources
- No promises outside policy
- Safe fallback message

Not exhaustive: a teaching map of common types. Real systems combine several, chosen by risk.

Guardrails in action

A support assistant for a fake shop. Each case runs twice: without the guardrail, then with it.

Open the interactive notes ↗

01 · OVERVIEW

One loop, three guards

Personal data, a refund rule and a secret filter, in one assistant.

02 · INPUT

Stopped at the door

Huge messages, card numbers, injection attempts, off-topic requests and threats.

03 · EXECUTION

Limits on actions

Roles that can't delete, refunds on others' orders, large refunds sent for approval, loops cut by a budget.

04 · OUTPUT

Checked before sending

Broken formats, leaked secrets, invented numbers and false promises, with a safe fallback.

Watch where each check sits, and what reaches the customer with and without it.

GUARDRAILS · ACCELERATORS

You don’t have to build every guardrail yourself

Open-source libraries and cloud services cover the common checks. Your business rules still need your own code.

ACCELERATOR	TYPE	WHAT IT GIVES YOU
NVIDIA NeMo Guardrails	Open source	Programmable rails for topics, dialogue flows, input and output checks
Guardrails AI	Open source	A hub of ready-made validators: personal data, toxicity, format, competitors…
Meta Llama Guard · Prompt Guard	Open models	Classifiers for unsafe content and for injection and jailbreak attempts
LLM Guard (Protect AI)	Open source	Input and output scanners: injection, secrets, personal data, toxicity
Microsoft Presidio	Open source	Detects and anonymises personal data in text and images
OpenAI Moderation · Agents SDK guardrails	API · framework	Harmful-content classification; input and output guardrails around agents
Azure AI Content Safety · Amazon Bedrock Guardrails · Google Model Armor	Cloud services	Managed filters: harmful content, prompt attacks, personal data, denied topics

Examples, not endorsements. Check fit, cost, latency and where your data is processed.

Main takeaways

1. **Guardrails are code, not requests.**
   The prompt guides the model; checks around it enforce the rules.

2. **Layer them: input, actions, output.**
   No single check catches everything, and the riskiest moment is when the model acts.

3. **Every guardrail has a cost.**
   Latency, false alarms and upkeep: test guardrails like the rest of the system.

LESSON

Evals overview

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

How do you know the system is good, and still good after a change? Understand what evals are, the main types, how a model can act as a judge, and how evals turn opinions into evidence.

Evals: repeatable tests of what the model produces

An eval takes an output, applies a check, and returns a result you can record and compare over time.

WHY

Reading doesn't scale

You can read ten answers, not ten thousand, and not after every change.

Thousands of conversations

WHY

Fixes break other things

A new instruction fixes refunds and quietly breaks returns.

Regressions

WHY

"Better" needs a number

Without a baseline, improvement is an impression.

Pass rate: 72% → 91%

WHY

Everything keeps changing

New models, prompts, tools and data: each one needs re-testing.

Model upgrade

No eval, no evidence. Evals decide whether a change ships, which model to use, and whether a guardrail works.

Three ways to check: code, a judge, people

FAST AND EXACT

Code checks

Rules a program can verify.

- Exact match, keywords
- Format and schema
- Tool-call rules
- Cost and latency

READS LIKE A PERSON

LLM as a judge

A second model grades against a rubric.

- Groundedness
- Correctness
- Completeness
- Policy compliance
- Tone
- Refusal
- Agent steps
- A vs. B

THE REFERENCE

Human review

Experts set what "good" means.

- Expert labels
- Checking the judge
- Spot checks in production

WHEN THEY RUN

Before release
experiments, regression tests

Inside the system
judge, then retry or fall back

In production
samples of live traffic

Not exhaustive: a teaching map. Use code where code works; a judge for what needs reading.

Evals in action

The same fake bike shop, from one judged answer to a full experiment.

Open the interactive notes ↗

01 · LLM AS A JUDGE

From opinion to a verdict

Keyword checks fail; a judge with a rubric gives a pass or fail, with the reason.

02 · WHAT JUDGES CHECK

And where they fail

Eight checks, and four traps: position, length, injection and noise. Then the judge is checked against people.

03 · JUDGE IN THE LOOP

Fail, then fix

Rewrite with the critique, switch to a stronger model, or stop with a safe answer.

04 · EXPERIMENT

Which prompt is better?

Same cases, several variants, and statistics that show how much of the difference is real.

Look at the numbers, then read the failures: they show what to change.

EVALS · ACCELERATORS

Tools for evals and LLM judges

They run your test cases, call the judge, store the results and compare versions. The dataset and the rubric are still yours to write.

ACCELERATOR	TYPE	WHAT IT GIVES YOU
promptfoo	Open source	Test cases in a config file; compare prompts and models; red-teaming
DeepEval	Open source	Unit-test style evals with ready-made LLM-judge metrics
Ragas	Open source	Metrics for RAG: faithfulness, relevance, context quality
Inspect AI (UK AI Security Institute)	Open source	A framework for rigorous evals, including agents and safety
OpenAI Evals	Open source · API	Eval templates and graders for OpenAI models
MLflow	Open source	Evaluation and LLM judges inside experiment tracking
LangSmith · Langfuse · Arize Phoenix · Braintrust	Platforms	Datasets, judges, experiments and production traces in one place

Examples, not endorsements. Many also cover observability, the next unit.

Main takeaways

1. **An eval turns “it seems better” into evidence.**
   Fixed cases, fixed checks, and a baseline to compare against.

2. **Code first, a judge second, people to calibrate.**
   A judge is a model too: measure it against human labels.

3. **Test the cases that matter, segment by segment.**
   An average can hide failures in the hard and adversarial cases.

NEXT · OBSERVABILITY

Evals tell you it works before release. Observability shows what happens in production.

Unit 07: traces, metrics and feedback from real users.

07. Observability

You can't improve what you can't see: what the system did, why, and at what cost.

What and why | Traces | Metrics and cost | Quality signals | Improvement loop | Tools | Takeaways

ONE IDEA TO KEEP

Measure what the system does, not how it feels.

Data, not impressions: in testing and in production.

LESSON

Observability and continuous improvement

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

A GenAI system can look fine and still be wrong, slow or expensive. Understand traces, metrics, cost and quality signals, how they show what to fix, and how they feed a loop of continuous improvement.

Observability: see what the system did, and why

Observability: see every step, its time and its cost. Traceability: follow any answer back to its question, sources, tools and version.

WHY

Wrong answers look right

A failed answer is fluent and confident. No error is raised.

No error, wrong policy

WHY

Many moving parts

Model, prompt, search, tools, agent: any of them can be the cause.

Where did it go wrong?

WHY

Cost and speed drift

Tokens, retries and loops add up, request by request.

The bill doubled in a week

WHY

Everything changes

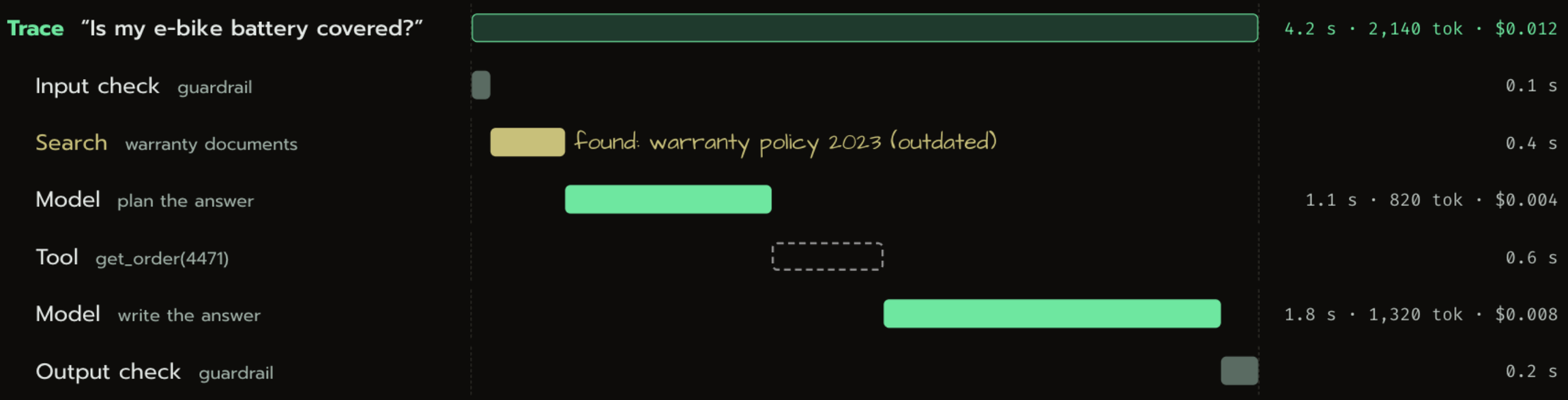
New models, prompts, documents and users, all the time.

Since the last release...

A healthy server can still give a wrong answer. Observability shows what happened; evals and people say whether it was right.

A trace: the story of one request

Every step of one answer, in order, with its time, tokens and cost.



A LOG

```
2026-11-02-09-14-03 INFO model call ok 1.8s
```

One event, on its own. Fine for one server; blind to the whole answer.

A TRACE

```
trace → search → model → tool → model
```

Every step of one request, linked: what went in, what came out, how long, how much.

Trace one request · **Span** one step · **Session** one conversation · **Tags** user, prompt and model version

Four signals to watch: steps, speed, cost, quality

WHAT HAPPENED?

Traces

- Steps, in order
- Inputs and outputs
- Sources and tools used
- Prompt and model version

IS IT WORKING?

Speed and errors

- Response time
- Errors and timeouts
- Tool failures
- Volume of requests

WHAT DOES IT COST?

Cost · FinOps

- Tokens per request
- Cost per user and feature
- Which model, for what
- Budgets and alerts

IS IT RIGHT?

Quality

- Eval scores
- LLM-as-a-judge scores
- User feedback 👍 👎
- Human review

The first three show the system is running. Only the fourth shows it is right.

Quality in production: score real answers

Live traffic
Every conversation is traced

→

Sample
e.g. 5% of answers, plus all 👎

→

Score
Code checks and an LLM judge

→

Dashboard
Scores over time, by version

→

Review queue
People check the worst cases

BEFORE RELEASE · UNIT 06

Evals on fixed test cases

Known questions, known good answers. Decides whether a change ships.

IN PRODUCTION · THIS UNIT

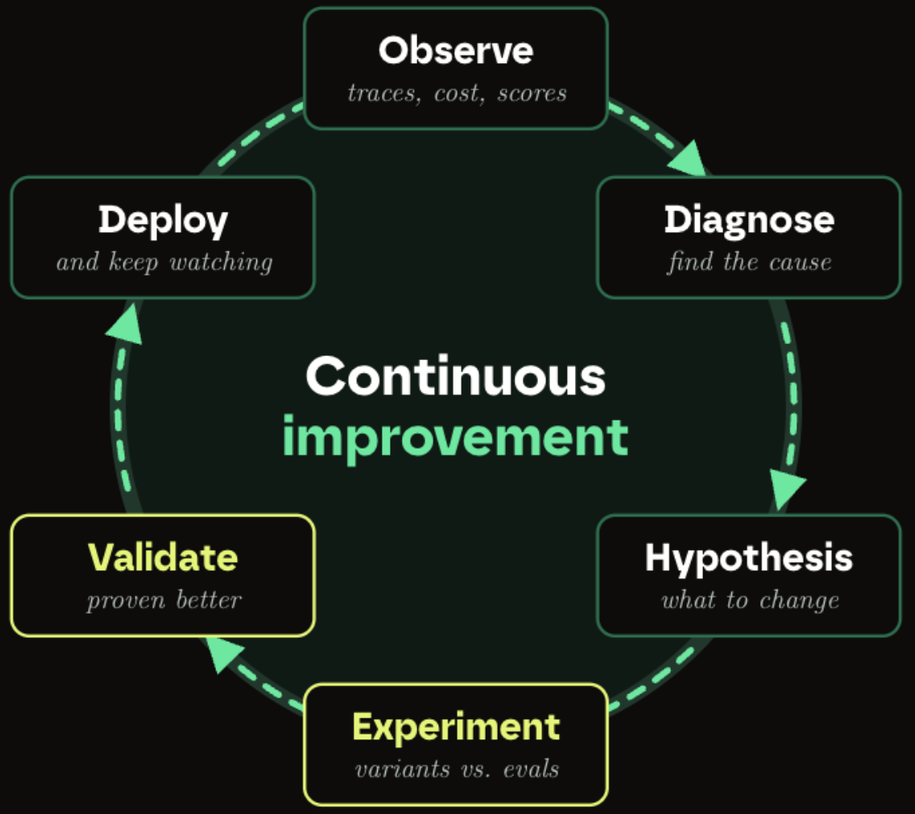
Evals on live samples

Real questions, no answer key: judge scores and user feedback. Finds drift and new failures.

WHAT GETS MEASURED

Groundedness · Correctness · Task completed · Tone · Safety · Response time · Cost

From observing to improving



1. **Every failure becomes a test case.**
   Production traces feed the eval dataset.
2. **Fix the right part.**
   Prompt, context, search, tools, agent or model: rarely retraining.
3. **Ship what is proven.**
   Each change goes through an experiment first.

Experiment and validate · **Unit 08**

Observability in action

The fake bike shop again: slow answers, a wrong reply, a doubled bill.

Open the interactive notes ↗

01 · ONE TRACE

Find the wrong answer

Follow one reply step by step: the search returned last year's warranty policy.

02 · SPEED AND COST

Where the money goes

An agent loop and a large model for simple questions double the cost per conversation.

03 · QUALITY

Scores drop after a release

A judge scores live answers; groundedness falls right after prompt v12.

04 · THE FIX, MEASURED

Change, test, compare

Failures become test cases; v13 wins on quality, cost and speed.

Each tab starts from a complaint and ends with evidence.

OBSERVABILITY · TOOLS

Tools for AI observability

They collect traces, add up tokens and cost, run judges on live answers and put it all on dashboards. Deciding what "good" means is still your job.

TOOL	TYPE	WHAT IT GIVES YOU
Langfuse	Open source · cloud	Traces, cost, prompts, evals and user feedback; can run on your own servers. Part of ClickHouse since 2026
LangSmith (LangChain)	Platform	Tracing, monitoring and evals; the natural fit for LangChain and LangGraph
Arize Phoenix	Source-available	Tracing and evals built on OpenTelemetry; strong on RAG
Braintrust	Platform	Eval-first: experiments, datasets and production logs side by side
Opik (Comet)	Open source	Tracing, LLM judges and evaluation of live traffic
MLflow Tracing	Open source	Traces and LLM judges inside the MLflow experiment tracker
W&B Weave	Platform	Tracing and evals for teams already on Weights & Biases
Datadog LLM Observability	Monitoring platform	LLM traces, cost and quality next to the rest of your IT monitoring
OpenTelemetry · GenAI conventions	Open standard	A shared format for AI traces, so you can switch tools later; still maturing

Examples, not endorsements. Most also run evals and experiments. Check cost, and where your data is stored.

Main takeaways

1. **Measure, don’t guess.**
   Decide with data, not feelings: in testing and in production.

2. **Running is not the same as right.**
   Combine traces, speed and cost with evals, judges and human feedback.

3. **Close the loop.**
   Failures become test cases, then experiments, then proven changes: continuous improvement.

NEXT · EXPERIMENTS

Observability tells you what to change. Experiments prove that the change works.

Unit 08: hypotheses, variants, A/B tests and decisions.

08. Experiments

Change one thing, measure it, and decide with evidence.

What and why | What to test | Offline to online | Datasets and evals | Results to insight | Tools | Takeaways

ONE IDEA TO KEEP

Every change is a hypothesis until the data says otherwise.

Test before production; decide with evidence, not with a demo.

LESSON

Experiments: test before production

DEPTH

- *Overview*
- *Conceptual*
- *Technical*
- *Deep dive*

OBJECTIVE

A new prompt, model, tool or agent can make things better, or quietly worse. Understand how to compare versions on the same cases and checks, what you can test, and how to turn results into a decision.

Experiments: compare before you ship

Run two or more versions on the same test cases, score them with the same evals, and compare.

WHY

Small changes, big effects

One sentence in a prompt can change thousands of answers.

Missing facts

WHY

Fixes break other things

Better refunds, worse returns: you only see it if you test both.

Regressions

WHY

Every gain has a price

Better answers can cost more or take longer.

+8% quality, +40% cost

WHY

Demos mislead

Three good examples prove nothing about the next thousand.

"It worked when I tried it"

Change one thing, keep the rest fixed, decide with numbers. That is all an experiment is.

Six steps, the same every time

1 Hypothesis
What should improve, and how you will measure it.
"v13 answers warranty questions better"

2 Dataset
Test cases with the expected outcome.
120 cases, 5 segments

3 Variants
The baseline and the change.
A = prompt v12 · B = prompt v13

4 Run
Every case through every variant.
240 answers

5 Evals
Code checks, an LLM judge, people.
pass rate, groundedness, cost

6 Decide
Ship, iterate or drop.
B wins: ship it

Kept fixed: cases, settings, judge and metrics, chosen **before** the run

Write the hypothesis and the metric first. Choosing them after seeing the results turns an experiment into a story.

Test the system, not just the model

PROMPTS

Instructions

- Which wording gets more right answers?
- Do examples help?

MODELS

Model and settings

- Can a smaller model do this job?
- Which provider, which settings?

CONTEXT

Context and RAG

- Which chunks, how many sources?
- Does reranking help?

TOOLS

Tools

- Does a better description stop wrong calls?
- One tool or three?

ORCHESTRATION

Agents and workflows

- Fixed workflow or agent?
- How many steps, what budget?

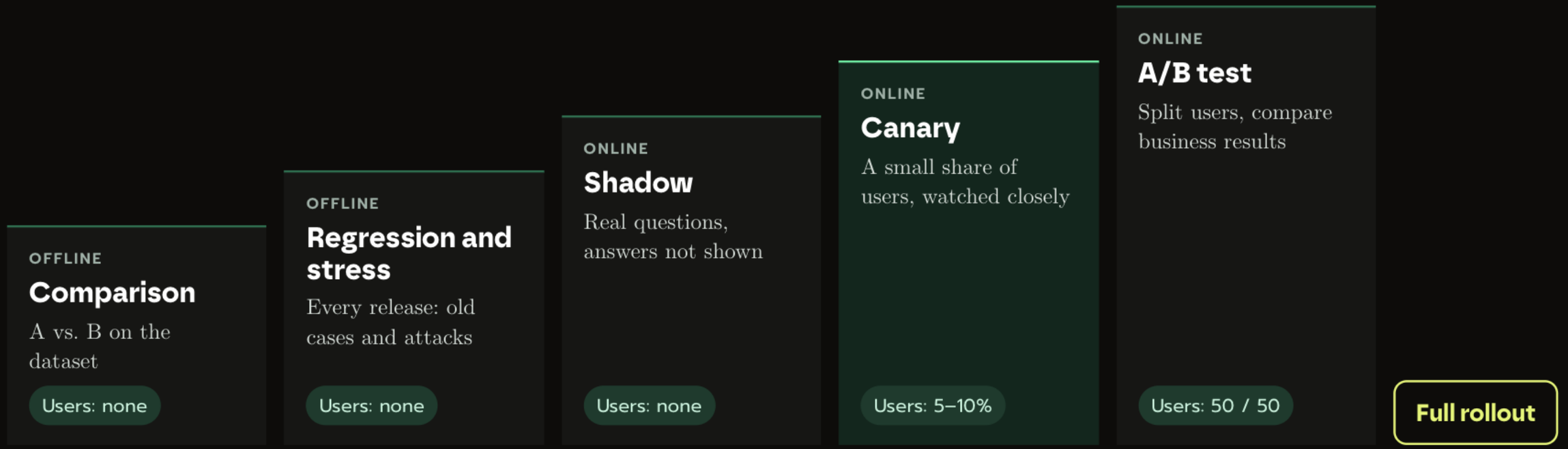
SAFETY

Guardrails

- How many attacks are stopped?
- How many good requests are blocked?

Each card is a variant you can put against the baseline, on the same dataset.

From the lab to real users, one step at a time



Also: an ablation removes one part, e.g. the reranker. If quality doesn't drop, you may not need it.

The dataset is the yardstick

WHERE CASES COME FROM

Experts
the questions that matter

Production failures
from observability, Unit 07

Synthetic
generated by an LLM, checked by people

Adversarial
red-team and edge cases

DATASET · V4

Question → expected outcome

"Return a helmet after 20 days?" → refuse: 14-day limit

Easy | Policy | Hard | Adversarial

Segments, so an average can't hide a weak spot

HOW ANSWERS ARE SCORED

Code checks
format, required facts, tool calls

LLM as a judge
grounded, correct, right action

People
calibrate the judge

Scoreboard
per variant, per segment

A good dataset is small, versioned, and grows with every failure.

From results to a decision

1. **Better than the baseline?**
   On the metric chosen before the run.
2. **In every segment?**
   An average can hide a drop in the hard cases.
3. **Is the difference real?**
   Enough cases, repeated runs, a judge checked by people.
4. **At what cost and speed?**
   Quality, cost and response time together.
5. **What do the failures say?**
   Read them: they point to the next change.

THEN DECIDE

Ship
Wins on the main metric; no segment worse; cost acceptable

Iterate
Promising, but fails a segment or costs too much

Drop
No real difference: keep the simpler, cheaper version

Experiments in action

Harbor Cycles tests a new prompt, a smaller model and an agent before release.

Open the interactive notes ↗

01 · DATASET AND EVALS

The yardstick

Cases by segment and source, with the checks that score them.

02 · PROMPTS AND MODELS

Which version wins?

Three variants; the small model is cheap but fails the attacks.

03 · AGENT AND TOOLS

Offline, then canary

A step budget stops loops; 10% of users confirm it.

04 · INSIGHTS

What to look at

Segments, real differences, trade-offs, failures, decision.

Press "Run experiment", then read the Insights tab.

EXPERIMENTS · TOOLS

Tools for experiments

They store datasets, run every variant, call the judges and show the results side by side. The hypothesis, the cases and the decision are still yours.

TOOL	TYPE	WHAT IT GIVES YOU
Braintrust	Platform	Datasets, experiments and side-by-side diffs; can block a release that scores worse
LangSmith (LangChain)	Platform	Datasets, experiments and LLM judges; natural fit for LangChain and LangGraph
Langfuse	Open source · cloud	Datasets and experiment runs, linked to production traces
promptfoo	Open source	Test cases in a config file; compare prompts and models; red-teaming. OpenAI announced its acquisition in 2026
DeepEval · Ragas	Open source	Ready-made metrics; Ragas focuses on RAG
Inspect AI (UK AI Security Institute)	Open source	Rigorous evals of agents and safety
MLflow · W&B Weave · Arize Phoenix	Open source · platforms	Experiments and evals inside ML tooling
GrowthBook · LaunchDarkly	Feature flags · A/B tests	Canary and A/B rollouts with real users

Examples, not endorsements. Most also cover evals and observability.

Main takeaways

1. **Test before production.**
   Every change to prompts, models, tools or agents is a hypothesis.

2. **Fix the dataset and the evals; change one thing.**
   Look at segments and read the failures: an average hides them.

3. **Decide on trade-offs, then keep watching.**
   Quality, cost and speed together; experiments and observability make continuous improvement.

09. Bonus: Cycles of Improvement

Improvement is not a project; it is three loops running at once.

The four dimensions | Three cycles | How they connect | Takeaways | Levels of AI adoption

ONE IDEA TO KEEP

A GenAI system improves through loops, not launches.

Each loop turns evidence into the next change.

Improving the Performance of GenAI Systems

The four dimensions don't work alone: they connect through three feedback cycles.

1 Model Training and Adaptation

- Pre-training (and continued pre-training)
- Post-training (RLHF and other post-training techniques)

Parametric memory

2 Harness and Capability Extension

- Memory systems and artifacts
- Context and knowledge integration
- Capabilities and tools
- Orchestration and control
- Execution and evaluation

3 Experiments and Validation

(offline)

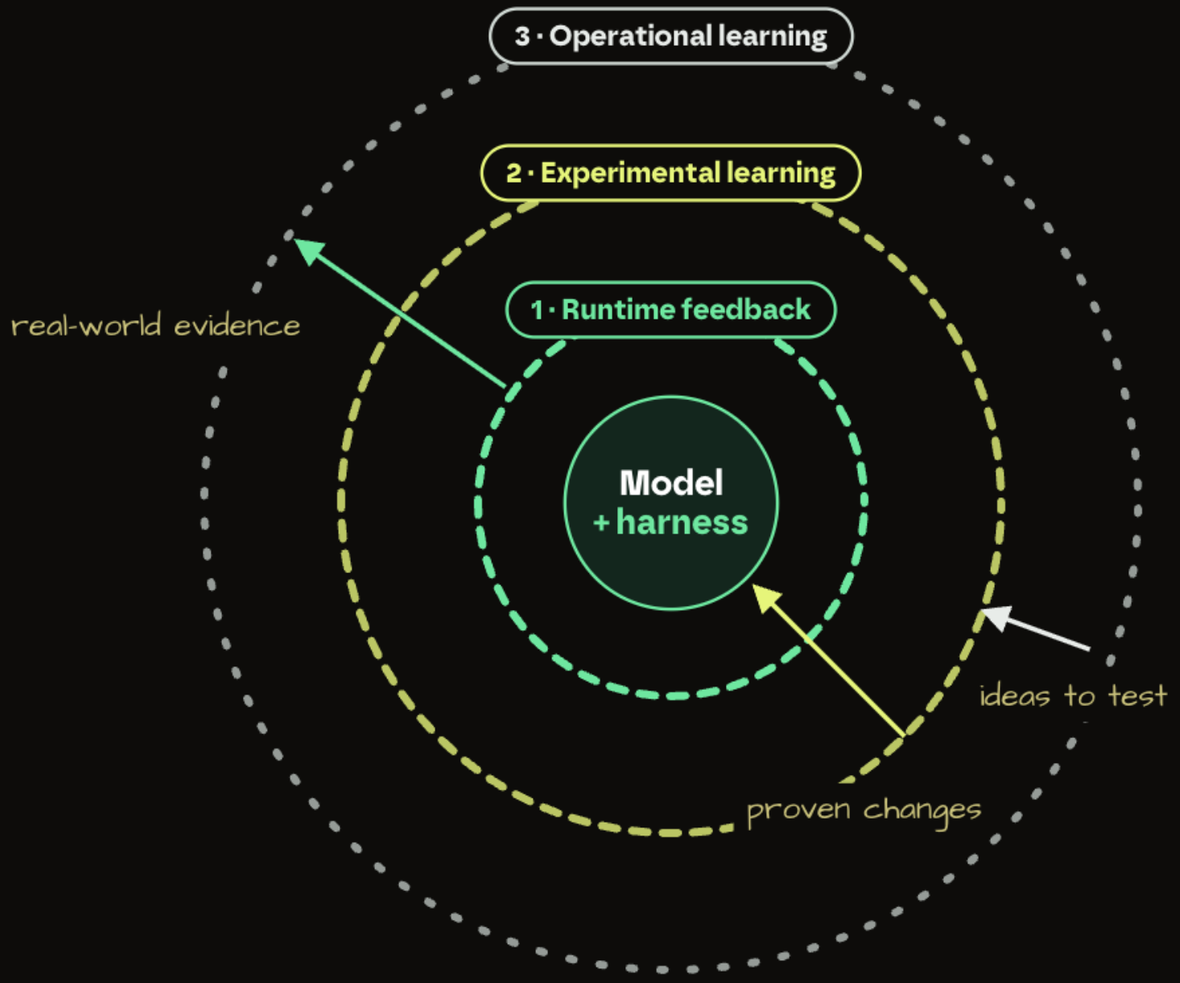
- Benchmarks
- Datasets
- Test cases
- Metrics
- Evaluations

4 Observability and Continuous Improvement

- Telemetry
- Traces
- Metrics and evaluators
- User feedback

CYCLES OF IMPROVEMENT

Three cycles, one system



Runtime feedback

ONE TASK · SECONDS

Checks and corrects the work while it runs.

Experimental learning

ONE CHANGE · DAYS

Tests a change and keeps it only if it is better.

Operational learning

WHOLE SYSTEM · WEEKS

Finds patterns in real use and sets what to fix next.

Real use → **ideas to test** → experiments → **proven changes** → model or harness → new evidence → …

The three cycles at a glance

CYCLE 1 · ONE TASK · SECONDS

Runtime feedback

Act → Check → Adjust

"Is this task on track?"

Retry, search again, ask a person for approval, or stop: within set limits.

A tool fails → the system recovers instead of giving up.

CYCLE 2 · ONE CHANGE · DAYS

Experimental learning

Idea → Test → Decide

"Is this change really better?"

Benchmarks, side-by-side tests, regression checks and human review, then adopt, reject or dig deeper.

Search misses answers → compare two search strategies.

CYCLE 3 · WHOLE SYSTEM · WEEKS

Operational learning

Collect → Patterns → Prioritize

"Where should we improve next?"

Traces, user feedback, quality scores, cost and speed, across many runs and versions.

Quality drops after a prompt change → open an experiment.

Sources: inner agent loop and outer improvement loops (Anthropic, *Building effective agents*, 2024); data flywheel from production traces to evals (H. Husain & S. Shankar, AI evals guidance, 2025–26).

Main takeaways

Correct inside every task

A good system checks its own work as it goes: it retries, searches again or asks a person, within limits you set.

Seconds · one task

Test every change before it ships

Experiments turn opinions into evidence. Keep a change only when it proves better on the same cases.

Days · one change

Let real use set the priorities

Production evidence shows where to improve next. Improving one part alone doesn't guarantee a better system.

Weeks · the whole system

TO CLOSE

Observe in production, experiment before release: that loop is how GenAI systems improve.

Evidence in, validated change out, again and again.

CYCLES OF IMPROVEMENT · WHERE YOU ARE

Six levels, L0 to L5

AI adoption isn't binary. Like self-driving cars, it moves in levels, and the higher you go, the more the improvement cycles must run on their own.

AUTOMATING WORKFLOWS

L0
AI as a Tool

Personal convenience; nothing reorganizes.

L1
Individual Automation

One person automates their own work.

L2
Team Automation

A team shares the workflow.

AGENTS IN YOUR ORG

L3
Directed Autonomy

You point an agent across the company.

L4
Self-Directed Autonomy

The agent notices and proposes.

L5
Self-Driving Enterprise

The agent acts; humans govern.

No fully self-driving organization exists yet.

Adapted from Ann Miura-Ko (Floodgate), six levels of enterprise AI adoption, modelled on the SAE levels of driving automation.

10. Project: Your GenAI System

One real problem, designed as a system, not a prompt.

The brief | Five limits | Checklist | Deliverables | Next class

Design a GenAI system for a real problem

Pick one bounded process or set of tasks, one user group and one measurable outcome. Show what changes, where GenAI fails, and how your system copes.

1

Data & Knowledge

Does the model have the right, current and proprietary knowledge?

Context

2

Context & Memory

What is available and kept during a run?

Context · Harness

3

Processing & Reasoning

Can it interpret, plan, use tools and execute?

Harness

4

Security, Safety & Control

Does it stay inside authorized boundaries?

Guards

5

Evaluation, Observability & Explainability

Can we measure, detect and diagnose failures?

Evals · Observability

GenAI fails, often silently: outputs can look right and still be wrong. Design for that.

Checklist, deliverables and next class

Checklist

- [ ] **Problem & baseline**
  No starting point, no proof of improvement.
- [ ] **Why GenAI**
  A rule or process change may be simpler.
- [ ] **Context**
  What the model must know, and from where.
- [ ] **Harness**
  Workflow or agent, tools, where a human decides.
- [ ] **Guards**
  What the system must never do or say.
- [ ] **Evals & observability**
  Metrics, test cases, what you log.

Deliverables

1. **System dossier**
   6–10 pages.
2. **Pitch**
   5 slides, 7 minutes.
3. **One-page system card**
   Problem, KPIs, top risks and guards, next decision.
4. **Small prototype**
   A toy that shows a few key concepts, not every spec or use case.

For next class

1. **A problem**
   Who, which task, why it matters.
2. **Main GenAI blocks**
   e.g. RAG, tools, workflow or agent, guards.
3. **Context, high level**
   Sources and data the system needs.

Full rules in the project PDF.